
ФАКУЛЬТЕТ	Робототехника и комплексная автоматизация (РК)
КАФЕДРА	Системы автоматизированного проектирования (РК6)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПISКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
«Разработка образовательной платформы с учебными курсами»

Студент	<u>РК6-41М</u>		<u>А.А. Онюшев</u>
	(группа)	(подпись)	(инициалы, фамилия)
Руководитель ВКР		(подпись)	<u>Ф.А. Витюков</u>
			(инициалы, фамилия)
Нормоконтролер		(подпись)	<u>С.В. Грошев</u>
			(инициалы, фамилия)

УТВЕРЖДАЮ

Заведующий кафедрой

РК6

(индекс)

А.П.Карпенко

(подпись)

12 февраля 2026г.

(дата)

З А Д А Н И Е

на выполнение выпускной квалификационной работы

Студент группы: РК6-41М

Онюшев Артем Андреевич

(фамилия, имя, отчество)

Тема выпускной квалификационной работы:

«Разработка образовательной платформы с учебными курсами»

При выполнении ВКР:

	Используются / Не используются	Да/Нет
1)	Литературные источники и документы, имеющие гриф секретности	Нет
2)	Литературные источники и документы, имеющие пометку «Для служебного пользования», иных пометок, запрещающих открытое опубликование	Нет
3)	Служебные материалы других организаций	Нет
4)	Результаты НИР (ОКР), выполняемой в МГТУ им. Н.Э. Баумана	Нет
5)	Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати	Нет

Тема квалификационной работы утверждена распоряжением по факультету:

Название факультета:

РК

Дата и рег. номер
распоряжения:

Часть 1. Аналитическая часть

Провести анализ современных образовательных платформ, определить состав их основных функциональных возможностей, пользовательских ролей и требований к безопасности, масштабируемости и удобству сопровождения.

Часть 2. Исследование систем аутентификации, авторизации и разграничения доступа
Провести анализ существующих подходов к аутентификации, авторизации и управлению доступом. Разработать модель доступа, применимую к образовательной платформе.

Часть 3. Разработка собственной системы авторизации и разграничения доступа
Разработать систему авторизации с помощью Ory Hydra. Разработать систему управления доступами с помощью Ory Keto.

Часть 4. Проектирование структуры данных и ключевых компонентов платформы
Разработать структуру доменной модели, базы данных и схему взаимодействия компонентов системы. Описать основные сущности платформы, их связи и пользовательские сценарии.

Часть 5. Реализация подсистем видео, онлайн-эфиров, уведомлений и чатов
Реализовать хранение и доставку видеоконтента, модуль проведения онлайн-эфиров, механизмы обмена сообщениями и отправки уведомлений. Интегрировать необходимые внешние и внутренние сервисы.

Оформление квалификационной работы:

Расчетно-пояснительная записка на 76 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.):
Слайды: 1 – Тема ВКР, 2 – Определения, 3 – Актуальность работы, 4 – Постановка задачи,
5 – Архитектура проекта, 6 – Механизмы авторизации, 7 – Реализация авторизации,
8 – Ролевая модель, 9 – Структура курса, 10 – Структура эфира, 11 – Видеоплеер,
12 – Кодеки, 13 – Демонстрация работы, 14 – Заключение.

Дата выдачи задания «10» февраля 2026 г.

В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до 2 июня 2026 г.

Руководитель квалификационной работы	_____	Ф.А. Витюков	10.02.2026
	(подпись)	(инициалы, фамилия)	(дата)
Студент	_____	А.А. Онюшев	10.02.2026
	(подпись)	(инициалы, фамилия)	(дата)

Примечание: Задание оформляется в двух экземплярах: один выдается обучающемуся, второй хранится на кафедре.

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ
КАФЕДРА
ГРУППА

РК
РК6
РК6-41М

УТВЕРЖДАЮ

Заведующий кафедрой
(подпись)

РК6
(индекс)
А.П. Карпенко
12 февраля 2026г.
(дата)

**КАЛЕНДАРНЫЙ ПЛАН
выполнения выпускной квалификационной работы**

Студента Онюшева Артема Андреевича
(фамилия, имя, отчество)

Тема квалификационной работы: «Разработка образовательной платформы с учебными курсами»

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		План	Факт	Должность	ФИО, подпись
1.	Задание на выполнение работы. Формулирование проблемы, цели и задач работы	11.02.2026	_____	Руководитель ВКР	Витюков Ф.А.
2.	1 часть. Аналитическая часть 1	18.02.2026	_____	Руководитель ВКР	Витюков Ф.А.
3.	Утверждение окончательных формулировок решаемой проблемы, цели работы и перечня задач	28.02.2026	_____	Заведующий кафедрой	Карпенко А.П.
4.	2 часть. Аналитическая часть 2	05.03.2026	_____	Руководитель ВКР	Витюков Ф.А.
5.	3 часть. Практическая часть 1	26.03.2026	_____	Руководитель ВКР	Витюков Ф.А.
6.	4 часть. Практическая часть 2	21.04.2026	_____	Руководитель ВКР	Витюков Ф.А.
7.	5 часть. Практическая часть 3	23.05.2026	_____	Руководитель ВКР	Витюков Ф.А.
8.	1-я редакция работы	28.05.2026	_____	Руководитель ВКР	Витюков Ф.А.
9.	Подготовка доклада и презентации	07.06.2026	_____	Студент	Дурнев П.А.
10.	Отзыв руководителя	07.06.2026	_____	Руководитель ВКР	Витюков Ф.А.
11.	Допуск работы к защите на ГЭК (нормоконтроль)	05.06.2026	_____	Нормоконтролер	Грошев С.В.
12.	Внешняя рецензия	07.06.2026	_____	Секретарь ГЭК	
13.	Защита работы на ГЭК	08.06.2026	_____	Секретарь ГЭК	

Руководитель квалификационной работы _____ Ф.А. Витюков 10.02.2026
(подпись) (инициалы, фамилия) (дата)

Студент _____ А.А. Онюшев 10.02.2026
(подпись) (инициалы, фамилия) (дата)

АННОТАЦИЯ

Данная выпускная квалификационная работа посвящена исследованию и разработке образовательной платформы с курсами, предназначенной для размещения учебных материалов, воспроизведения видеоуроков, проведения онлайн-эфиров, организации общения между пользователями и управления доступом к контенту. Актуальность темы обусловлена ростом интереса к цифровым образовательным сервисам, необходимостью обеспечения надежного хранения и доставки медиаконтента, а также повышенными требованиями к безопасности, масштабируемости и удобству сопровождения современных веб-платформ.

В рамках работы рассматриваются архитектурные и технологические решения, применимые к созданию современной образовательной платформы. Особое внимание уделяется вопросам аутентификации, авторизации и разграничения прав доступа, поскольку в рамках одной системы должны корректно взаимодействовать различные категории пользователей, включая учеников, авторов курсов и администраторов. Для реализации механизмов доступа предполагается использование специализированных решений ORY Hydra и ORY Keto, позволяющих построить гибкую и масштабируемую модель авторизации.

В работе также рассматриваются вопросы проектирования структуры данных и ключевых компонентов платформы. Платформа должна обеспечивать хранение информации о курсах, уроках, пользователях, ролях, материалах и эфирах, а также поддерживать сценарии взаимодействия, связанные с доступом к видеоконтенту, проведением прямых трансляций, отправкой уведомлений и работой чатов в режиме реального времени.

Практическая часть работы включает разработку основных подсистем платформы, связанных с хранением и воспроизведением видео, организацией онлайн-эфиров, отправкой уведомлений и обменом сообщениями между пользователями. В качестве технологической основы используются

современные серверные и инфраструктурные средства, ориентированные на построение надежных и расширяемых веб-систем.

Результатом работы является архитектурно обоснованная образовательная платформа с курсами, видеоуроками, онлайн-эфирами и системой разграничения доступа, а также выводы, подтверждающие применимость выбранных технологических и архитектурных решений для создания современных образовательных сервисов.

В расчетно-пояснительной записке 73 страниц, 37 рисунков, 1 листинг, 2 таблицы, 14 слайдов.

ОСНОВНЫЕ ПОНЯТИЯ И ОПРЕДЕЛЕНИЯ

Образовательная платформа – программная система, предназначенная для размещения, организации и предоставления учебного контента пользователям, а также для управления процессом обучения и взаимодействием участников.

Курс – логически завершенный набор учебных материалов, объединенных общей темой, структурой и целью обучения.

Урок – структурный элемент курса, содержащий отдельный фрагмент учебного материала в текстовом, видео- или смешанном формате.

Видеоурок – учебный материал, представленный в виде заранее подготовленной видеозаписи, доступной пользователю для просмотра.

Онлайн-эфир – формат синхронного взаимодействия пользователей в режиме реального времени, предполагающий проведение прямой трансляции учебного занятия или иного мероприятия.

Медиаконтент – совокупность цифровых материалов платформы, включая видеофайлы, изображения, документы, презентации и иные вложения.

Пользователь – зарегистрированный участник платформы, взаимодействующий с системой в соответствии со своей ролью и предоставленными правами доступа.

Роль – совокупность полномочий, определяющих доступ пользователя к определенным функциям и ресурсам платформы.

Ролевая модель доступа – формальное описание ролей пользователей и соответствующих им прав в системе.

Аутентификация – процесс проверки подлинности пользователя, подтверждающий его личность при входе в систему.

Авторизация – процесс определения перечня действий и ресурсов, доступных пользователю после успешной аутентификации.

Разграничение доступа – механизм предоставления или ограничения доступа пользователей к функциям и данным системы в зависимости от их роли и полномочий.

SSO (Single Sign-On) – механизм единого входа, позволяющий пользователю после одной процедуры аутентификации получать доступ к нескольким компонентам системы без повторного ввода учетных данных.

OAuth 2.0 – протокол авторизации, обеспечивающий делегированный доступ к защищенным ресурсам.

OpenID Connect – протокол аутентификации, построенный поверх OAuth 2.0 и позволяющий подтверждать личность пользователя.

JWT (JSON Web Token) — компактный формат токена, используемый для безопасной передачи данных о пользователе и его правах.

Backend – серверная часть программной системы, отвечающая за бизнес-логику, хранение данных, обработку запросов и интеграцию внешних сервисов.

API (Application Programming Interface) – программный интерфейс взаимодействия между различными компонентами системы.

PostgreSQL – реляционная система управления базами данных, используемая для хранения структурированной информации платформы.

Redis – in-memory хранилище данных, применяемое для кэширования, временного хранения информации и ускорения работы системы.

Kafka – распределенный брокер сообщений, используемый для организации событийного взаимодействия между компонентами системы.

Centrifugo – платформа для обмена сообщениями и доставки событий в режиме реального времени.

Объектное хранилище – способ хранения файлов и связанных с ними метаданных в виде объектов.

S3-совместимое хранилище – объектное хранилище, поддерживающее интерфейсы взаимодействия, совместимые с Amazon S3.

Видеосервис – специализированный программный сервис, предназначенный для хранения, обработки, публикации и воспроизведения видеоконтента.

Видеоплеер – программный компонент интерфейса, предназначенный для воспроизведения видеоматериалов пользователем.

Уведомление – сообщение, автоматически направляемое пользователю при наступлении определенного события в системе.

Чат – подсистема платформы, предназначенная для обмена сообщениями между пользователями в реальном времени.

Масштабируемость – способность системы сохранять работоспособность и приемлемую производительность при росте числа пользователей, объема данных и количества запросов.

Отказоустойчивость – способность системы продолжать функционирование при частичных сбоях отдельных компонентов.

Нефункциональные требования – требования к качественным характеристикам системы, включая производительность, надежность, безопасность и удобство сопровождения.

База данных (БД) – это организованная совокупность данных, которая хранится в электронном виде на компьютере или специальном сервере.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
1 Архитектура платформы	14
1.1 Frontend	15
1.2 Backend.....	16
1.3 PostgreSQL	17
1.4 Redis	18
1.5 Kafka.....	18
1.6 S3	19
1.7 Centrifugo	20
2 Системы аутентификации и авторизации	21
2.1 Обзор существующих SSO-решений.....	22
2.2 Практическое применение Ory Hydra.....	26
3 Управление доступом.....	29
3.1 Основные модели.....	31
3.2 Дискреционная модель доступа DAC.....	31
3.3 Мандатная модель доступа MAC.....	32
3.4 Списки контроля доступа ACL	33
3.5 Ролевая модель доступа RBAC	34
3.6 Атрибутивная модель доступа ABAC	35
3.7 Модель доступа на основе отношений ReBAC	37
3.8 Ory Keto	38
3.9 Настройка Ory Keto	39
3.10 Разработанная система.....	41
4 Видеокодеки	45
4.1 Сравнение видеокодек.....	45
5 Структура курса	50
5.1 Общая информация о курсе.....	50
5.2 Контакты до и после покупки курса.....	51
5.3 Чаты курса	52
5.4 Уроки курса	54

5.5	Модули курса	55
5.6	Защита видеоконтента.....	56
5.7	Запуск курса	57
6	Структура эфира	59
6.1	Общая информация о эфире	59
6.2	Параметры доступа эфира	60
6.3	Настройка поведения после завершения эфира	62
7	Описание видеоплеера	64
ЗАКЛЮЧЕНИЕ		67
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		70
ПРИЛОЖЕНИЕ А. Графическая часть выпускной квалификационной работы		72

ВВЕДЕНИЕ

В последние годы цифровые образовательные технологии получили широкое распространение и стали важной частью современной образовательной среды. Онлайн-обучение активно применяется в высшем образовании, корпоративной подготовке, системах повышения квалификации и авторских образовательных продуктах. Рост числа пользователей дистанционных сервисов приводит к повышению требований к качеству разработки образовательных платформ, их надежности, безопасности и удобству эксплуатации.

Современная образовательная платформа представляет собой сложную программную систему, объединяющую множество функциональных возможностей. К числу таких возможностей относятся регистрация и аутентификация пользователей, управление курсами и уроками, предоставление доступа к видеоурокам и дополнительным материалам, проведение онлайн-эфиров, обмен сообщениями, отправка уведомлений и администрирование контента. При этом платформа должна обслуживать различные категории пользователей, включая учеников, авторов курсов, модераторов и администраторов, каждая из которых обладает собственным набором прав и ограничений.

Одной из ключевых задач при разработке образовательной платформы является корректная организация доступа к данным и функциям системы. Пользователь должен иметь возможность работать только с теми ресурсами, которые соответствуют его роли и текущему контексту. Например, один пользователь может просматривать приобретенные курсы, другой — создавать и редактировать учебные материалы, а третий — управлять структурой платформы и ролями других пользователей. В связи с этим особую значимость приобретают механизмы аутентификации, авторизации и разграничения прав доступа.

Не менее важным аспектом является организация работы с медиаконтентом. Современные образовательные платформы активно используют видеоматериалы как основной способ передачи учебной

информации. Помимо заранее записанных видеоуроков, востребованным становится проведение прямых эфиров, обеспечивающих синхронное взаимодействие между преподавателем и аудиторией. Подобная функциональность предъявляет повышенные требования к хранению и доставке видеоконтента, а также к реализации чатов, уведомлений и других real-time механизмов.

При создании подобных систем необходимо не только реализовать пользовательский интерфейс и базовую серверную логику, но и обоснованно выбрать архитектурные и технологические решения. От этих решений зависят производительность системы, ее масштабируемость, удобство сопровождения, устойчивость к нагрузке и возможность дальнейшего развития. Недостаточно просто разработать платформу с курсами как набор страниц и форм. Требуется спроектировать комплексную систему, способную надежно работать в условиях роста числа пользователей, контента и сценариев взаимодействия.

Актуальность темы выпускной квалификационной работы обусловлена необходимостью разработки современной образовательной платформы, сочетающей средства управления курсами, хранения и воспроизведения видеоконтента, проведения онлайн-эфиров, отправки уведомлений и гибкого разграничения прав доступа. Исследование и разработка подобной системы представляют практический интерес, поскольку позволяют сформировать подход к построению образовательных веб-платформ, отвечающих современным требованиям безопасности, масштабируемости и удобства эксплуатации.

Объектом исследования являются образовательные веб-платформы, предназначенные для организации онлайн-обучения и предоставления учебного контента пользователям.

Предметом исследования являются архитектурные, технологические и программные решения, применяемые при разработке образовательной платформы с курсами, видеоуроками, онлайн-эфирами, уведомлениями, чатами и системой разграничения доступа.

Целью работы является исследование и разработка образовательной платформы с курсами, обеспечивающей управление пользователями, курсами, уроками, медиаконтентом и доступом к ресурсам системы.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

1. Провести анализ современных образовательных платформ и определить состав их основных функциональных возможностей, пользовательских ролей и требований к безопасности, масштабируемости и удобству сопровождения.
2. Исследовать существующие подходы к аутентификации, авторизации и управлению доступом в веб-системах.
3. Разработать модель доступа, применимую к образовательной платформе с различными пользовательскими ролями.
4. Разработать систему авторизации с использованием ORY Hydra и систему управления доступом с использованием ORY Keto.
5. Спроектировать структуру доменной модели, базы данных и схему взаимодействия компонентов платформы.
6. Описать основные сущности платформы, их связи и пользовательские сценарии.
7. Реализовать хранение и доставку видеоконтента, модуль проведения онлайн-эфиров, а также механизмы обмена сообщениями и отправки уведомлений.
8. Интегрировать внешние и внутренние сервисы, необходимые для функционирования платформы.

Практическая значимость работы заключается в разработке архитектурно обоснованной образовательной платформы, которая может быть использована как основа для дальнейшего развития реального программного продукта. Полученные результаты также могут быть полезны при проектировании аналогичных систем, в которых требуется сочетание образовательного контента,

видеоматериалов, онлайн-взаимодействия пользователей и гибкой модели разграничения доступа.

Структура работы определяется поставленной целью и задачами. В работе рассматриваются предметная область образовательных платформ, исследуются подходы к аутентификации и авторизации, описываются проектирование структуры данных и ключевых компонентов системы, а также реализация подсистем видео, онлайн-эфиров, уведомлений и чатов. В заключительной части формулируются основные результаты и выводы по выполненной работе.

1 Архитектура платформы

Разработанная образовательная платформа построена на основе распределенной сервисной архитектуры. Такой подход выбран из-за высокой функциональной неоднородности системы: платформа должна обеспечивать регистрацию и авторизацию пользователей, управление курсами и уроками, обработку видеоконтента, хранение файлов, проведение онлайн-эфиров, работу чатов, отправку уведомлений и проверку прав доступа. Объединение всей этой логики в одном монолитном приложении усложнило бы сопровождение, масштабирование и дальнейшее развитие системы.

В основе архитектуры лежит разделение ответственности между отдельными сервисами. Каждый сервис отвечает за собственную предметную область и взаимодействует с остальными компонентами через HTTP API, gRPC или событийную шину. Такой подход позволяет независимо развивать отдельные части платформы, масштабировать наиболее нагруженные компоненты и снижать связность между подсистемами.

На рисунке 1 представлены основные сервисы платформы и инфраструктурные компоненты: backend-сервисы на Golang, PostgreSQL, Redis, Kafka, Centrifugo, MinIO как S3-совместимое объектное хранилище, а также сервисы Ory Hydra и Ory Keto, отвечающие за авторизацию и проверку прав доступа.

настройку запуска курса. React хорошо подходит для таких интерфейсов, поскольку позволяет хранить состояние формы на клиентской стороне, валидировать введенные данные до отправки на сервер и последовательно отображать пользователю только актуальный шаг настройки.

Отдельное значение имеет работа React-приложения с real-time функциональностью. Для отображения уведомлений и сообщений чата frontend подключается к Centrifugo и получает события в режиме реального времени. Это позволяет пользователю без перезагрузки страницы видеть новые сообщения, системные уведомления, события эфиров и изменения состояния отдельных элементов платформы. Такой подход повышает интерактивность системы и делает пользовательский опыт ближе к современным образовательным и коммуникационным сервисам.

Также frontend отвечает за интеграцию пользовательского интерфейса с подсистемой видеоконтента. На стороне React реализуется отображение видеоплеера, элементов управления воспроизведением, выбора качества, изменения скорости, полноэкранного режима и дополнительных визуальных элементов. При этом доступ к видеоматериалам остается контролируемым backend-сервисами: клиент получает только те ссылки и параметры воспроизведения, которые разрешены пользователю после серверной проверки.

Таким образом, React в архитектуре платформы выполняет роль основного инструмента построения пользовательского интерфейса. Он обеспечивает компонентную структуру frontend-приложения, поддержку сложных форм и сценариев, взаимодействие с backend API, отображение real-time событий и работу с видеоплеером. Благодаря этому клиентская часть остается расширяемой, сопровождаемой и пригодной для дальнейшего развития функциональности платформы.

1.2 Backend

Серверная часть платформы реализуется на языке Golang. Выбор Golang обусловлен его пригодностью для разработки сетевых сервисов, высокой производительностью и удобной моделью конкурентного выполнения. Для

образовательной платформы это особенно важно, так как система одновременно обрабатывает пользовательские запросы, события курсов, загрузку файлов, работу чатов, уведомления и фоновые процессы обработки видео.

Дополнительным преимуществом Golang является простая модель конкурентности на основе `goroutine` и `channel`. Она позволяет эффективно реализовывать фоновые задачи, обработчики событий, `consumers` для Kafka и сервисы, работающие с внешними API. Часть операций платформы выполняется асинхронно: обработка видео, отправка уведомлений, обновление статусов, обработка событий курсов и эфиров. Для таких задач Golang хорошо подходит как по производительности, так и по удобству разработки.

Также Golang удобен для построения микросервисной архитектуры. Каждый сервис может быть собран в отдельный исполняемый файл и развернут независимо. Это упрощает деплой, мониторинг и масштабирование отдельных компонентов системы. Например, сервис обработки видео может масштабироваться отдельно от сервиса курсов, а сервис чатов — отдельно от сервиса авторизации.

1.3 PostgreSQL

В качестве основного хранилища структурированных данных используется PostgreSQL. Эта СУБД применяется для хранения информации о пользователях, курсах, уроках, модулях, эфирах, чатах, сообщениях, промокодах и других доменных сущностях платформы.

PostgreSQL подходит для данной системы, поскольку образовательная платформа оперирует большим количеством связанных сущностей. Например, курс связан с автором, модулями, уроками, файлами, видео, доступами и чатами. Для таких данных важны транзакционность, поддержка связей, индексов, ограничений целостности и возможность выполнять сложные выборки.

В архитектуре платформы каждый доменный сервис может иметь собственную базу данных PostgreSQL или отдельную схему. Такой подход соответствует принципу изоляции данных в микросервисной архитектуре: сервис управляет своей предметной областью и не должен напрямую изменять

данные другого сервиса. Взаимодействие между сервисами осуществляется через API или события, а не через прямые запросы к чужой базе данных.

1.4 Redis

Redis используется как in-memory хранилище для данных, к которым требуется быстрый доступ или которые имеют ограниченный срок жизни. В рамках образовательной платформы Redis может применяться для хранения временных состояний авторизации, одноразовых кодов, данных сессий, кэша часто запрашиваемой информации и промежуточных технических значений.

Использование Redis позволяет снизить нагрузку на PostgreSQL и ускорить обработку запросов. Например, при авторизации пользователя временный код подтверждения или состояние auth-flow не обязательно хранить в основной базе данных. Такие данные имеют короткий жизненный цикл и могут быть безопасно размещены в Redis с ограниченным временем хранения.

Кроме того, Redis может использоваться для ограничения частоты запросов, хранения временных блокировок и быстрой проверки служебных состояний. Это особенно полезно для сценариев входа, отправки SMS-кодов, защиты от частых повторных запросов и обработки временных пользовательских состояний.

1.5 Kafka

Для асинхронного взаимодействия между компонентами платформы используется Kafka. Событийная шина позволяет сервисам обмениваться сообщениями без жесткой синхронной зависимости друг от друга. Один сервис публикует событие, а другой сервис обрабатывает его в удобный момент через consumer.

Такой подход особенно важен для операций, которые не должны блокировать пользовательский сценарий. Например, после создания курса можно сформировать событие, которое будет обработано отдельным consumer для обновления индексов, отправки уведомлений или выполнения дополнительных фоновых действий. При загрузке видео сервис файлов или

course-сервис может отправить событие, которое затем будет обработано video-consumer для запуска обработки видео.

Kafka также используется для событий, связанных с курсами, мессенджером и видео. На схеме присутствуют отдельные event-хранилища и consumers, например course-consumer, video-consumer и stream-consumer. Это позволяет разгрузить основные API-сервисы и вынести длительные или повторяемые операции в фоновые обработчики.

Примерами событий в образовательной платформе могут быть: создание курса, публикация курса, добавление урока, загрузка видео, завершение обработки видео, начало эфира, завершение эфира, отправка уведомления, создание чата и изменение прав доступа.

1.6 S3

Для хранения файлов в архитектуре используется MinIO как S3-совместимое объектное хранилище. Такой подход подходит для хранения неструктурированных данных: видеофайлов, изображений, обложек курсов, материалов к урокам, документов, презентаций и иных вложений.

Использование объектного хранилища позволяет отделить тяжелые файловые данные от основной реляционной базы. PostgreSQL хранит метаданные файлов: идентификатор, владельца, тип, размер, статус, связь с уроком или курсом. Сам файл при этом размещается в MinIO. Это снижает нагрузку на базу данных и упрощает масштабирование хранилища.

Сервис files отвечает за загрузку, удаление и получение файлов. При загрузке файла он принимает запрос от клиента, сохраняет объект в MinIO, записывает метаданные в PostgreSQL и при необходимости формирует событие для других сервисов. Например, если загружен видеоматериал, может быть создано событие для запуска обработки видео.

S3-совместимый интерфейс делает такое решение гибким: при необходимости MinIO можно заменить на другое объектное хранилище, поддерживающее аналогичный API. Это снижает зависимость от конкретной реализации и упрощает возможную миграцию инфраструктуры.

1.7 Centrifugo

Для доставки сообщений и уведомлений в режиме реального времени используется Centrifugo. Он обеспечивает WebSocket-взаимодействие между серверной частью и клиентским приложением, позволяя отправлять пользователям события без постоянного опроса backend API.

Подсистема real-time взаимодействия используется для чатов курса, чатов эфира, уведомлений о событиях и оперативной доставки пользовательских сообщений. Например, когда пользователь отправляет сообщение в чат, backend проверяет его права, сохраняет сообщение и передает событие в канал Centrifugo. Остальные участники, подключенные к соответствующему каналу, получают сообщение в реальном времени.

Использование Centrifugo позволяет разгрузить backend-сервисы от необходимости самостоятельно поддерживать большое количество WebSocket-соединений. Основной backend отвечает за бизнес-логику и проверку прав, а Centrifugo выполняет доставку сообщений подключенным клиентам.

Сервис chat отвечает за управление чатами внутри платформы, а messenger-сервис используется для интеграций и управления внешними коммуникационными сценариями. Такая структура позволяет отделить внутренние чаты курса и эфира от внешних интеграций.

2 Системы аутентификации и авторизации

Разработка сайта для проведения обучающих курсов требует особого внимания к вопросам аутентификации и авторизации пользователей. Данные механизмы являются основой безопасности веб-приложения и напрямую влияют на устойчивость системы, корректность разграничения доступа и защиту персональных данных. В образовательных платформах данные вопросы приобретают дополнительную значимость, поскольку система обслуживает большое количество пользователей с различными ролями и уровнями доступа.

Аутентификация представляет собой процесс проверки подлинности пользователя, то есть подтверждение того, что пользователь действительно является тем, за кого себя выдаёт. Авторизация, в свою очередь, определяет, какие действия и ресурсы доступны аутентифицированному пользователю. В контексте обучающих платформ это может включать доступ к курсам, учебным материалам, административным функциям, результатам обучения и другим компонентам системы. На рисунке 2 показан наглядный пример сравнения авторизации и аутентификации.

В современных веб-приложениях аутентификация и авторизация редко реализуются в виде изолированного модуля. Как правило, они являются частью распределённой архитектуры и взаимодействуют с другими сервисами, что требует стандартизированных протоколов и хорошо продуманной архитектуры.

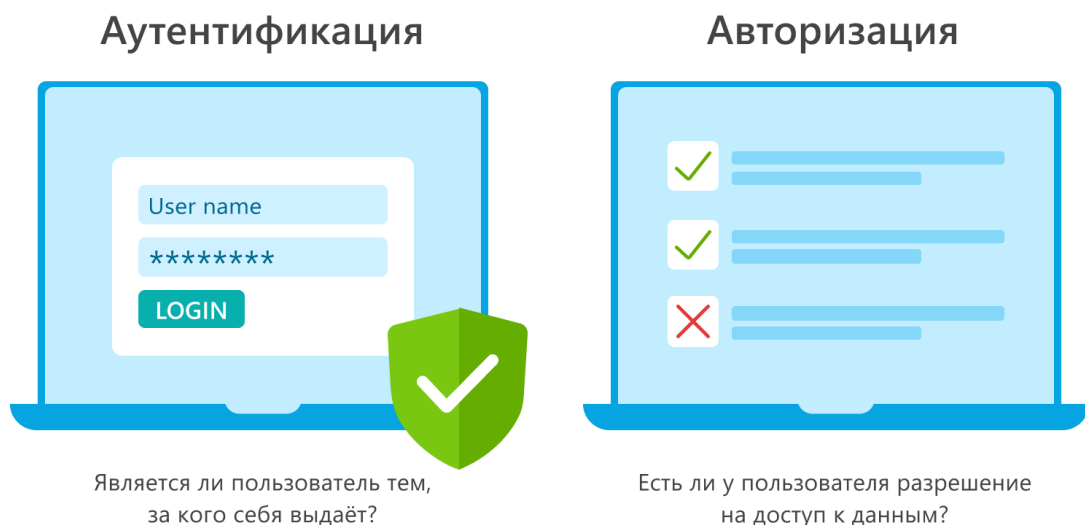


Рис. 2 Аутентификация и авторизация

2.1 Обзор существующих SSO-решений

Одним из распространённых подходов к организации аутентификации в сложных веб-системах является использование механизма единого входа (Single Sign-On, SSO). SSO позволяет пользователю пройти процедуру аутентификации один раз и затем получать доступ ко всем связанным сервисам без повторного ввода учетных данных.

В образовательных платформах использование SSO особенно актуально, поскольку такие системы часто состоят из нескольких компонентов, включая пользовательский интерфейс, административную панель, сервисы хранения данных, сервисы аналитики и интеграции с внешними системами. Повторная аутентификация при каждом обращении к отдельному компоненту снижает удобство использования и увеличивает вероятность ошибок. На рисунке 3 показана схема работы SSO.

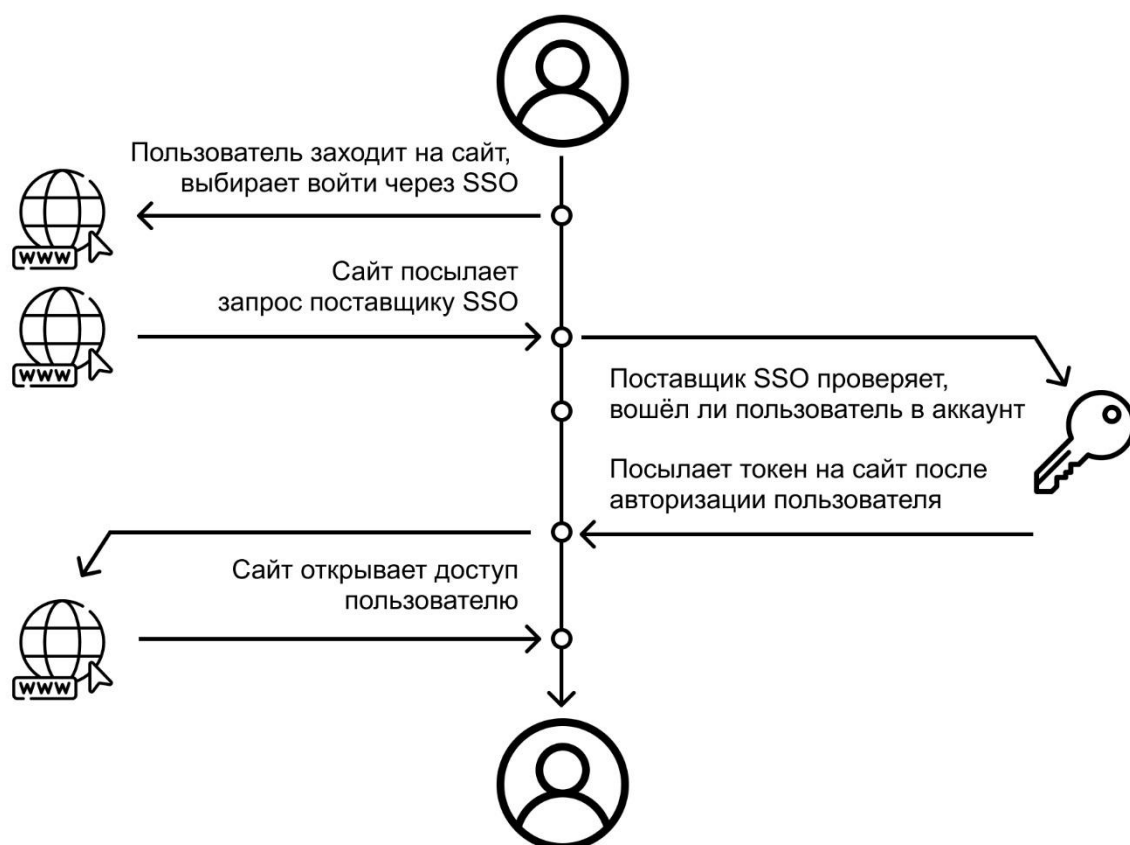


Рис. 3 схема работы SSO

На современном рынке представлено множество решений для реализации аутентификации и авторизации с поддержкой SSO. Эти решения различаются по

архитектуре, сложности внедрения, возможностям настройки и модели распространения. В рамках данной работы рассмотрены наиболее распространённые подходы и системы.

Одним из базовых стандартов в области аутентификации является протокол OAuth 2.0, который широко используется для делегирования доступа между сервисами. OAuth 2.0 определяет набор потоков авторизации, позволяющих приложениям получать доступ к защищённым ресурсам от имени пользователя. На основе OAuth 2.0 был разработан протокол OpenID Connect, который расширяет его возможности и добавляет механизм идентификации пользователя.

Использование OAuth 2.0 и OpenID Connect позволяет реализовать безопасную аутентификацию без передачи учетных данных сторонним сервисам. Однако данные протоколы представляют собой спецификации и требуют дополнительной реализации серверной части, что усложняет разработку при отсутствии готовой инфраструктуры.

Одним из популярных готовых решений является система Keycloak, представляющая собой сервер аутентификации с поддержкой OAuth 2.0, OpenID Connect и SAML. Keycloak предоставляет широкий функционал, включая управление пользователями, ролями и политиками доступа. В то же время данное решение обладает монолитной архитектурой и требует значительных ресурсов при масштабировании, что может быть критично для высоконагруженных систем.

Коммерческие облачные решения, такие как Auth0, предлагают готовую инфраструктуру для аутентификации и авторизации с минимальными затратами на внедрение. Основным недостатком таких решений является высокая стоимость при росте количества пользователей и зависимость от внешнего провайдера, что может ограничивать гибкость архитектуры и усложнять соблюдение требований по локализации данных.

Существуют также более простые решения, такие как Firebase Authentication, ориентированные на быстрое создание приложений. Однако

подобные системы плохо подходят для сложных сценариев разграничения доступа и микросервисных архитектур, характерных для образовательных платформ. На рисунке 4 представлены логотипы перечисленных SSO решений.



Рис. 4 Логотипы готовых SSO решений.

Альтернативой использованию готовых решений является разработка собственной системы аутентификации и авторизации. Данный подход позволяет полностью контролировать архитектуру и логику работы SSO, а также адаптировать систему под конкретные требования проекта.

При этом разработка собственной SSO-системы связана с рядом существенных сложностей. В первую очередь это вопросы безопасности, включая защиту учетных данных, корректную реализацию криптографических алгоритмов, защиту от атак повторного воспроизведения и утечек токенов. Ошибки в реализации могут привести к уязвимостям, устранение которых потребует значительных усилий.

Кроме того, собственная SSO-система требует постоянного сопровождения, обновления и аудита безопасности. При росте нагрузки возникает необходимость в оптимизации и масштабировании, что дополнительно увеличивает сложность поддержки. С учётом данных факторов разработка собственной системы аутентификации оправдана, как правило, только в крупных проектах с особыми требованиями.

Для выбора оптимального решения по реализации аутентификации и авторизации в разрабатываемой обучающей платформе был проведён сравнительный анализ различных подходов и существующих SSO-решений. Целью анализа являлось выявление решения, наиболее полно удовлетворяющего требованиям безопасности, масштабируемости, гибкости настройки и

экономической целесообразности. В таблице 1 приведен сравнительный анализ рассмотренных решений SSO.

Таблица 1 – Сравнительный анализ разных SSO

Решение SSO	Безопасность	Масштабируемость	Гибкость настройки	Внедрение	Поддержка	Стоимость
Keycloak	5	2	3	3	4	2
Auth0	5	4	3	5	5	4
Собственная	3	5	5	5	1	5
ORY	4	5	5	4	4	1

Анализ таблицы показывает, что монолитные решения, такие как Keycloak, обеспечивают приемлемый уровень безопасности, однако обладают ограниченной гибкостью и требуют значительных ресурсов при масштабировании. Облачные сервисы, такие как Auth0, демонстрируют высокий уровень безопасности и удобство внедрения, но характеризуются высокой стоимостью владения и зависимостью от внешнего провайдера.

Разработка собственной SSO-системы позволяет добиться максимальной гибкости, однако сопровождается высокими рисками, связанными с безопасностью, а также значительными трудозатратами на разработку и сопровождение. В условиях ограниченных ресурсов данный подход является экономически нецелесообразным.

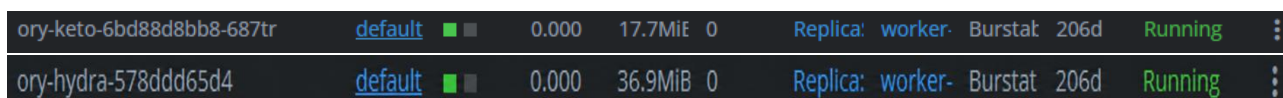
Экосистема ORY демонстрирует сбалансированные показатели по всем ключевым критериям. Модульная архитектура, соответствие стандартам OAuth 2.0 и OpenID Connect, а также ориентация на облачные и микросервисные среды делают данное решение оптимальным для использования в образовательной платформе. При этом сама система имеет открытый код, написанный на Golang, что удобно для проекта, написанного на этом языке.

По результатам проведённого анализа было принято решение использовать экосистему ORY для реализации аутентификации и авторизации. Данная экосистема включает специализированные сервисы, каждый из которых отвечает за отдельную функциональную область.

ORY Hydra используется для реализации OAuth 2.0 и OpenID Connect и отвечает за выпуск и валидацию токенов доступа. ORY Keto применяется для управления политиками доступа и реализации моделей разграничения прав.

Такое разделение ответственности позволяет построить гибкую и масштабируемую систему аутентификации, соответствующую требованиям микросервисной архитектуры.

Таким образом, выбор экосистемы Ory является обоснованным с точки зрения безопасности, масштабируемости и соответствия архитектурным требованиям разрабатываемой обучающей платформы. На рисунке 5 изображены запущенные поды Ory Keto и Ory Hydra в Kubernetes уже для итогового проекта.



ory-keto-6bd88d8bb8-687tr	default	0.000	17.7MiE	0	Replica: worker-	Burstat	206d	Running	⋮
ory-hydra-578ddd65d4	default	0.000	36.9MiB	0	Replica: worker-	Burstat	206d	Running	⋮

Рис. 5 Запущенные поды Ory Keto и Ory Hydra в Kubernetes

2.2 Практическое применение Ory Hydra

В разрабатываемой образовательной платформе Ory Hydra используется в качестве отдельного сервиса авторизации, отвечающего за реализацию протоколов OAuth 2.0 и OpenID Connect. Его применение позволяет вынести критически важную логику выдачи, обновления и проверки токенов из основного backend-приложения в специализированный компонент, предназначенный именно для построения безопасных auth-flow. Такой подход снижает риск ошибок при самостоятельной реализации механизма авторизации и делает систему более устойчивой с точки зрения безопасности и дальнейшего сопровождения.

Пользователь может выполнять вход как через платформенный сервис авторизации, так и потенциально через сторонние поставщики идентификации. Для этого используется сценарий авторизации с получением authorization code и последующим обменом этого кода на JWT-токены.

На первом этапе клиентская часть приложения инициирует процесс входа пользователя. Перед отправкой запроса формируются параметры state и PKCE. Параметр state используется для защиты от CSRF-атак и позволяет проверить, что ответ авторизационного сервера соответствует именно тому запросу, который был создан клиентом. Механизм PKCE применяется для

дополнительной защиты authorization code: даже если код авторизации будет перехвачен, злоумышленник не сможет обменять его на токены без исходного code_verifier.

После генерации state и PKCE клиент отправляет пользователя на страницу авторизации. В зависимости от выбранного сценария входа, запрос может быть обработан платформенным сервисом или внешним поставщиком идентификации. Пользователь проходит процедуру аутентификации. После успешной проверки auth-сервис возвращает authorization code, который далее используется для получения JWT-токенов.

Следующим этапом является обработка полученного кода авторизации. Backend или клиентское приложение отправляет authorization code в Ory Hydra и выполняет обмен кода на набор токенов. В результате успешного обмена пользователь получает access token, а также при необходимости refresh token и id token. Access token используется при обращении к защищенным backend API платформы. Id token может применяться для получения информации об аутентифицированном пользователе, а refresh token – для продления пользовательской сессии без повторного ввода учетных данных.

На рисунке 6 данный процесс представлен, как последовательность операций: сначала выполняется генерация PKCE и state, затем отправляется запрос на получение кода авторизации, после чего производится обработка полученного кода и получение JWT-токенов. В ситуации, когда токены не были получены, система выводит ошибку и возвращает пользователя на этап авторизации. Это необходимо для того, чтобы исключить продолжение работы приложения в неопределенном состоянии, когда личность пользователя не подтверждена или сессия не была корректно создана.

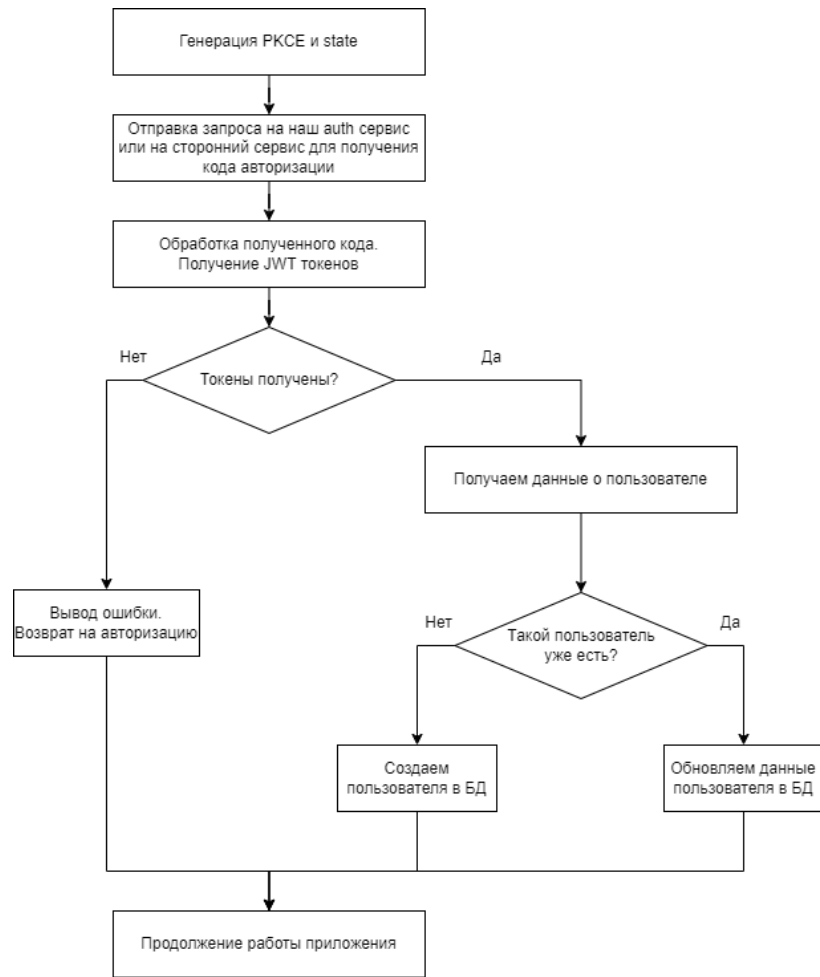


Рис. 6 Блок-схема процесса авторизации пользователя

3 Управление доступом

В современных программных системах управление доступом является одной из базовых составляющих общей архитектуры безопасности. Независимо от предметной области приложения, будь то корпоративная система, облачная платформа, банковский сервис, документооборот или образовательный портал, необходимо определять, какие пользователи могут выполнять те или иные действия и к каким данным они имеют доступ. Такая задача решается с помощью механизмов аутентификации, авторизации и разграничения прав доступа.

Аутентификация представляет собой процесс проверки подлинности пользователя, то есть подтверждение того, что пользователь действительно является тем, за кого себя выдает. Обычно аутентификация осуществляется с помощью логина и пароля, одноразовых кодов, токенов, биометрических данных или иных механизмов подтверждения личности. Однако факт успешной аутентификации сам по себе еще не означает, что пользователю разрешено выполнять любые действия в системе.

После прохождения аутентификации вступает в действие авторизация. Авторизация определяет, какие функции и ресурсы доступны конкретному пользователю. Например, в одной и той же системе один пользователь может только просматривать документы, другой – редактировать их, а третий – управлять правами доступа других пользователей. Таким образом, авторизация отвечает за проверку допустимости совершаемого действия.

Разграничение доступа является более широким понятием и охватывает правила, механизмы и модели, которые используются для назначения, хранения и проверки прав пользователей. По сути, это способ формального описания того, кто, что и при каких условиях может делать в системе. Корректно реализованное разграничение доступа позволяет не только защищать данные и функции приложения, но и поддерживать целостность бизнес-логики.

Необходимость систем управления доступом обусловлена несколькими причинами. Во-первых, разные категории пользователей работают с системой по-разному и требуют различного уровня полномочий. Во-вторых, с ростом

сложности информационной системы увеличивается количество объектов, к которым требуется ограничивать доступ: документы, записи баз данных, проекты, разделы, курсы, уроки, файлы и другие сущности. В-третьих, в современных приложениях часто требуется учитывать не только роль пользователя, но и его отношение к конкретному объекту. Например, один и тот же пользователь может иметь право редактировать собственный курс, но не иметь права редактировать курс другого автора.

В простейших системах разграничение доступа может быть реализовано вручную через условные проверки в коде. Однако такой подход плохо масштабируется, затрудняет сопровождение и приводит к дублированию логики в различных частях приложения. По этой причине в реальных системах применяются формализованные модели доступа, которые позволяют централизованно описывать и проверять права пользователей.

Системы управления доступом должны удовлетворять ряду требований. Прежде всего, они должны обеспечивать безопасность, то есть исключать несанкционированный доступ к данным и функциям. Также они должны быть гибкими, чтобы поддерживать изменение ролей, появление новых сущностей и усложнение предметной области. Кроме того, важны прозрачность и управляемость: разработчик или администратор должен понимать, по каким правилам пользователю предоставляется или запрещается доступ. Наконец, для современных распределенных систем важна масштабируемость, поскольку проверка прав доступа должна оставаться корректной и эффективной даже при большом количестве пользователей и объектов.

Таким образом, системы управления доступом представляют собой важнейший элемент современной информационной системы. Они обеспечивают реализацию бизнес-правил, защиту данных и корректное взаимодействие пользователей с приложением. Для выбора и построения эффективного механизма разграничения прав необходимо рассматривать различные модели доступа и оценивать их применимость в зависимости от особенностей проектируемой системы.

3.1 Основные модели

В информационных системах применяются различные модели управления правами доступа, отличающиеся способом описания разрешений, степенью гибкости и областью применения. Выбор конкретной модели зависит от типа системы, сложности предметной области, количества пользователей, структуры данных и требований к безопасности. Одни модели хорошо подходят для простых иерархических систем, другие ориентированы на строгое централизованное управление, а третьи используются в современных распределенных веб-приложениях, где доступ зависит не только от роли пользователя, но и от его связи с конкретным объектом.

В рамках практики были рассмотрены основные модели управления доступом: DAC, MAC, ACL, RBAC, ABAC и ReBAC. Каждая из них имеет собственную область применения, а также свои преимущества и ограничения.

3.2 Дискреционная модель доступа DAC

DAC, или Discretionary Access Control, представляет собой дискреционную модель управления доступом, при которой решение о предоставлении доступа к объекту принимается владельцем этого объекта. Иными словами, пользователь, создавший или контролирующий ресурс, может самостоятельно определить, кто и в каком объеме может с ним работать.

Подобный подход является одним из наиболее простых и исторически ранних. Он широко применялся в файловых системах, где владелец файла может предоставлять или ограничивать доступ другим пользователям. Примером может служить назначение прав на чтение, запись и выполнение файла в операционной системе. На рисунке 7 представлена модель доступа DAC.

Объект Субъект	O ₁	O ₂	O ₃	O ₄ (Printer)
S ₁	read			
S ₂				Print
S ₃		Read	Execute	
S ₄	read write		read write	

Рис.7 Модель доступа DAC

Преимуществом DAC является простота понимания и реализации. Пользователь получает возможность самостоятельно управлять принадлежащими ему ресурсами, что удобно в небольших и относительно изолированных системах. Однако при усложнении архитектуры приложения такая модель становится менее удобной. Пользователи начинают независимо назначать права, что затрудняет централизованный контроль и может приводить к противоречивым или небезопасным конфигурациям доступа.

Таким образом, DAC хорошо подходит для базовых сценариев управления объектами, но ограниченно применим в системах, где требуется строгая централизованная политика безопасности.

3.3 Мандатная модель доступа MAC

MAC, или Mandatory Access Control, представляет собой мандатную модель управления доступом, при которой правила доступа задаются централизованно и не могут произвольно изменяться пользователями. В отличие от DAC, здесь пользователь не является владельцем политики доступа к ресурсу. Все права определяются системой на основании заранее заданных правил, уровней допуска и классификаций объектов.

Такая модель применяется в системах, где особенно важны строгий контроль и предотвращение несанкционированного распространения информации. Примерами являются военные, государственные и иные защищенные информационные системы, где каждому субъекту и объекту могут быть присвоены уровни секретности, а доступ разрешается только при соблюдении установленных условий. На рисунке 8 представлена модель доступа MAC.

Мандатное модель доступа

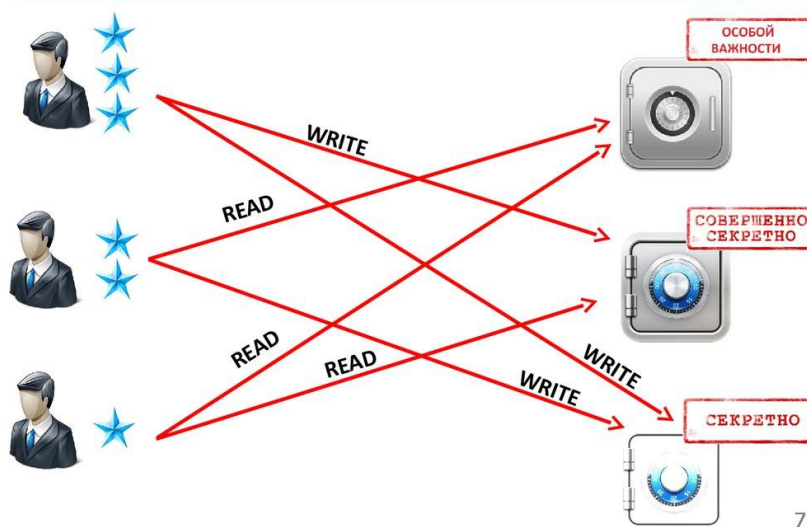


Рис.8 Модель доступа MAC

Главным преимуществом MAC является высокий уровень формального контроля и предсказуемости политики безопасности. Пользователь не может самовольно изменить параметры доступа, что уменьшает риск ошибок и злоупотреблений. Однако такая модель сложна в сопровождении и недостаточно гибка для типовых прикладных веб-систем. В пользовательских приложениях, корпоративных порталах и образовательных платформах необходимость в столь жестком централизованном механизме возникает редко.

Следовательно, MAC представляет интерес как фундаментальная модель безопасности, но в практике разработки прикладных систем используется ограниченно.

3.4 Списки контроля доступа ACL

ACL, или Access Control List, представляет собой способ задания прав доступа через список разрешений, связанный с конкретным объектом. В рамках такого подхода у каждого ресурса хранится перечень субъектов и допустимых для них действий. Например, для документа может быть определено, какие пользователи имеют право просматривать, изменять или удалять его.

Формально ACL не всегда рассматривается как отдельная глобальная модель наравне с RBAC или ABAC, однако в прикладной разработке это один из

наиболее распространенных механизмов хранения и проверки прав. Он нагляден и удобен для объектов, к которым требуется назначать индивидуальный доступ.

Основное достоинство ACL заключается в прозрачности. Для каждого объекта можно явно увидеть, кто имеет к нему доступ. Это удобно в системах с небольшим числом объектов и пользователей. Однако по мере роста системы возникают сложности: списки становятся длинными, их трудно сопровождать, а права начинают дублироваться между множеством объектов. Кроме того, ACL слабо подходит для описания более сложной логики, когда права зависят от роли пользователя, его отдела, принадлежности к группе или отношений между сущностями.

Таким образом, ACL удобен как локальный механизм описания прав на объект, но в больших системах требует сочетания с более общей моделью управления доступом. На рисунке 9 представлена модель доступа DAC.



Рис.9 Модель доступа ACL

3.5 Ролевая модель доступа RBAC

RBAC, или Role-Based Access Control, является одной из наиболее известных и широко используемых моделей разграничения доступа. В ее основе лежит идея назначения прав не напрямую пользователям, а ролям. Пользователь, в свою очередь, получает определенную роль, а вместе с ней и соответствующий набор разрешений.

Например, в информационной системе могут существовать роли «администратор», «редактор», «преподаватель», «студент», «модератор». Для каждой роли задается свой набор допустимых действий. Пользователю достаточно назначить подходящую роль, после чего система автоматически определит его полномочия.

Преимуществом RBAC является ее простота, формальная определенность и удобство администрирования. Эта модель особенно эффективна в системах, где набор функций хорошо делится по типовым ролям. Вместо назначения прав каждому пользователю отдельно администратор управляет ограниченным числом ролей, что уменьшает сложность конфигурации.

Однако у RBAC есть и ограничения. В простейшей форме эта модель не учитывает контекст доступа и не отражает отношения пользователя с конкретным объектом. Например, роль «автор» может означать право редактировать курсы, но сама по себе не показывает, какие именно курсы пользователь вправе изменять. В результате в сложных системах приходится дополнять RBAC дополнительными проверками в коде или вводить большое количество специальных ролей, что снижает прозрачность модели.

Таким образом, RBAC остается удобной и практичной моделью для большого числа прикладных систем, но в чистом виде не всегда достаточна для описания сложных объектно-ориентированных прав доступа. На рисунке 10 представлена модель доступа RBAC.

РОЛЕВАЯ МОДЕЛЬ ДОСТУПА (RBAC)

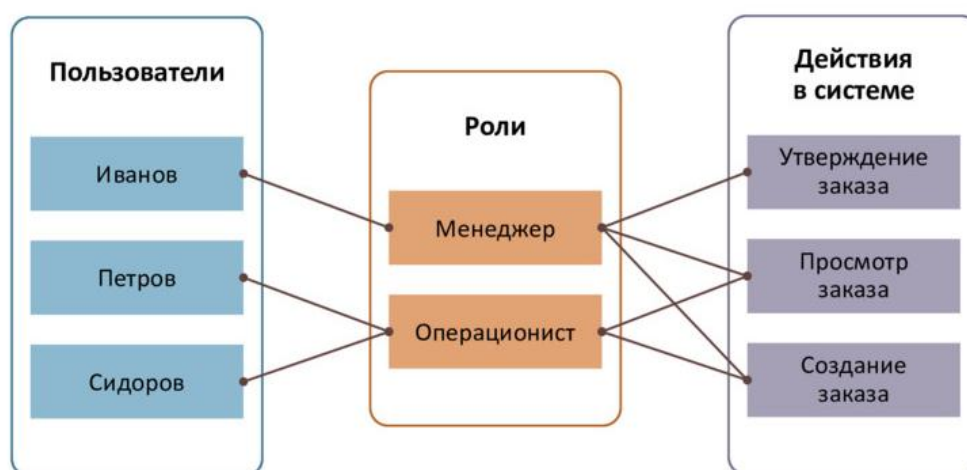


Рис. 10 Модель доступа RBAC

3.6 Атрибутивная модель доступа ABAC

ABAC, или Attribute-Based Access Control, представляет собой модель управления доступом, в которой решение о предоставлении прав принимается на

основе набора атрибутов. Атрибуты могут относиться к пользователю, объекту, действию или контексту выполнения запроса.

Например, при использовании АВАС доступ может определяться не только ролью пользователя, но и его подразделением, регионом, статусом, временем обращения, принадлежностью объекта к конкретной категории и иными свойствами. Условие доступа в таком случае задается в виде логического правила. Например: пользователь может просматривать документ, если он работает в том же отделе, что и владелец документа, и документ не имеет ограниченного уровня доступа.

Главным преимуществом АВАС является высокая гибкость. Эта модель позволяет формализовать сложные правила доступа без чрезмерного роста числа ролей. Она особенно полезна в системах, где доступ должен учитывать множество параметров одновременно.

Однако обратной стороной гибкости является повышенная сложность. Политики АВАС труднее проектировать, проверять и сопровождать. При большом числе атрибутов и правил возрастает риск возникновения сложной и плохо читаемой конфигурации, которую трудно анализировать и объяснять администраторам и разработчикам.

Следовательно, АВАС подходит для систем с развитой политикой безопасности и большим количеством контекстных условий, но требует более высокой дисциплины проектирования. На рисунке 11 представлена модель доступа АВАС.

АВАС: СХЕМА ОРГАНИЗАЦИИ ДОСТУПА



Рис. 11 Модель доступа АВАС

3.7 Модель доступа на основе отношений ReBAC

ReBAC, или Relationship-Based Access Control, представляет собой модель разграничения доступа, в которой права определяются через отношения между субъектами и объектами. В данной модели доступ пользователя зависит не только от его роли или атрибутов, но и от того, как он связан с конкретным ресурсом.

Например, в системе пользователь может быть владельцем курса, участником группы, редактором документа, подписчиком проекта или модератором чата. Доступ в таком случае задается не общим правилом вида «все преподаватели могут редактировать все курсы», а более точным отношением: «пользователь может редактировать только тот курс, владельцем или редактором которого он является».

ReBAC особенно удобна для современных прикладных систем, в которых объекты образуют сложные связи: пользователи состоят в командах, группы имеют участников, курсы содержат модули, модули содержат уроки, а права могут наследоваться по иерархии. Такая модель хорошо отражает логику реальных бизнес-процессов и позволяет описывать доступ более естественным образом.

Сильной стороной ReBAC является ее выразительность. Вместо создания большого числа искусственных ролей можно описывать реальные отношения

между сущностями. Это делает модель особенно полезной для платформ с большим количеством объектов и зависимых прав. Недостатком является более высокая концептуальная сложность по сравнению с RBAC. Для проектирования и сопровождения такой системы требуется более аккуратное описание доменной модели и правил наследования прав.

Таким образом, ReBAC является одним из наиболее перспективных подходов для современных веб-систем, особенно в тех случаях, когда доступ тесно связан со структурой предметной области. Данная модель представлена на рисунке 12.

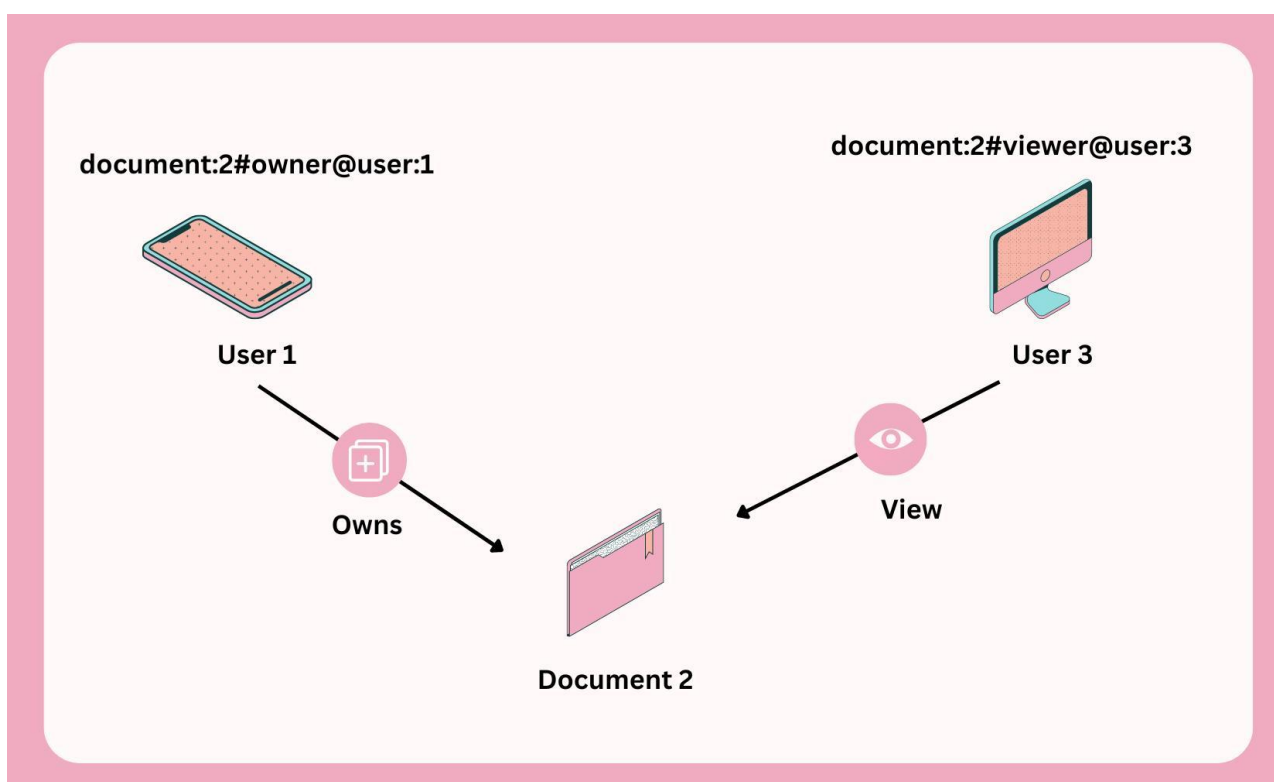


Рис. 12 Модель доступа на основе отношений ReBAC

3.8 Ory Keto

После рассмотрения основных моделей управления правами доступа особый интерес представляет практическая реализация relationship-based подхода. Для этой цели был изучен инструмент ORY Keto, предназначенный для построения гибкой системы авторизации и разграничения доступа.

ORY Keto представляет собой специализированный сервис управления доступом, предназначенный не для хранения пользовательских учетных данных и не для аутентификации, а именно для проверки прав доступа к объектам

системы. Его задача состоит в том, чтобы отвечать на вопросы вида: может ли данный пользователь просматривать, редактировать или удалять конкретный объект. В документации ORY подчеркивается, что отношения являются базовым типом данных Ory Permissions и описывают связи между объектами и субъектами, которые можно рассматривать как ребра графа.

Такой подход позволяет вынести проверку прав из бизнес-логики приложения в отдельный компонент. В результате система становится более прозрачной, а сама логика авторизации может описываться в формализованном виде, а не в виде множества разрозненных условных проверок в коде.

ORY Keto особенно полезен в тех случаях, когда доступ зависит не только от роли пользователя, но и от его связи с конкретным объектом. Например, пользователь может быть владельцем курса, участником группы, редактором документа или модератором эфира. Для подобных сценариев relationship-based модель оказывается более естественной, чем простое назначение ролей. На рисунке 13 изображен логотип Ory Keto.



Рис. 13 Логотип Ory Keto

3.9 Настройка Ory Keto

После изучения теоретических основ relationship-based модели доступа следующим этапом стало ознакомление с базовой настройкой ORY Keto и рассмотрение примеров его использования. Цель данного этапа состояла в том, чтобы понять, каким образом описывается модель доступа, как задаются отношения между сущностями и как на практике выполняется проверка разрешений.

Согласно официальной документации ORY, Ory Keto Quickstart строится вокруг примера видеосервиса, где права доступа определяются через отношения владельца и права просмотра, а сами отношения описываются в виде relation tuples. Такой пример демонстрирует, как Keto используется в роли permission

server, а приложение выступает в роли клиента, отправляющего запросы на проверку доступа. На рисунке 14 приведен пример из официальной документации.

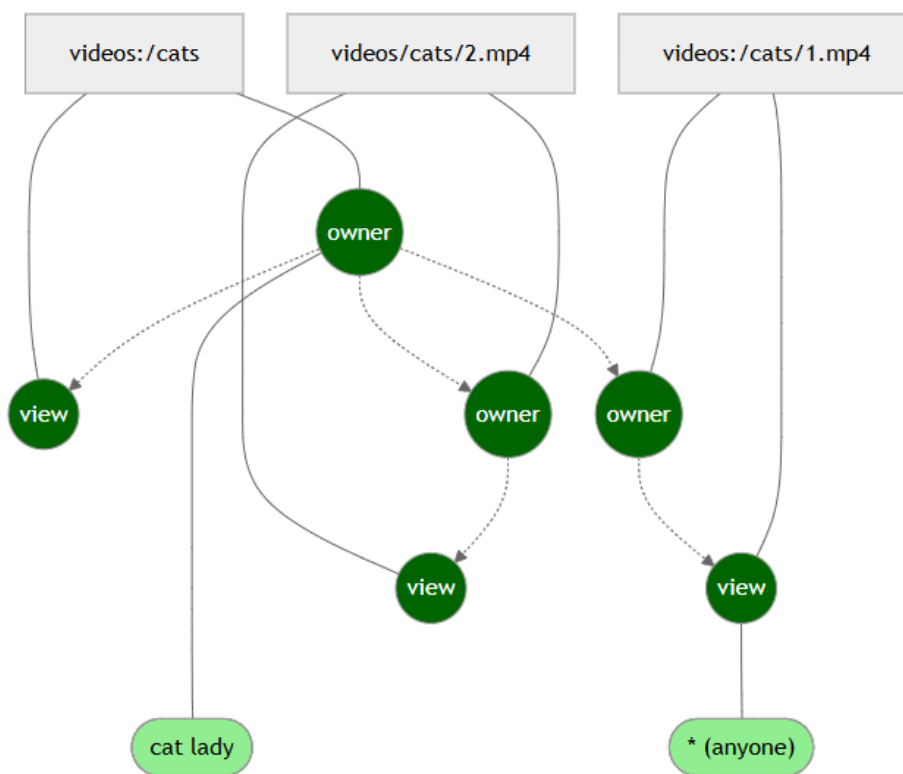


Рис.14 Пример от Ory Keto

Для запуска self-hosted ORY Keto используется конфигурационный файл, обычно называемый keto.yml. В нем задаются параметры подключения к базе данных, адреса read и write API, а также путь к файлу с описанием модели доступа.

В листинге 1 приведен пример базовой конфигурации keto.yml, который подходит для локального запуска и учебного примера с видеосервисом.

Листинг 1 – Пример конфигурации

```
1 dsn: memory/db_url
2
3 serve:
4   read:
5     host: 0.0.0.0
6     port: 4466
7   write:
8     host: 0.0.0.0
9     port: 4467
10
11 log:
12   level: info
```

13	format: text
14	
15	namespaces:
16	- id: 0
17	name: User
18	- id: 1
19	name: Video

В данном примере:

- dsn: memory означает использование встроенного in-memory хранилища для учебного запуска;
- serve.read задает read API, через которое выполняются проверки доступа;
- serve.write задает write API, через которое создаются и изменяются отношения;
- namespaces указывает текущие области имен для модели доступа.

Такой вариант удобен именно для учебного или демонстрационного стенда, поскольку не требует отдельной базы данных. Для реальной эксплуатации вместо memory обычно используется постоянное хранилище, например PostgreSQL.

3.10 Разработанная система

Процесс взаимодействия пользователя с системой начинается со страницы входа, на которой предусмотрена возможность авторизации по номеру телефона или адресу электронной почты с использованием пароля. Интерфейс страницы разработан с учетом требований минимализма и удобства восприятия, что снижает вероятность ошибок при вводе данных и повышает удобство использования. На рисунке 15 изображен интерфейс формы входа.

Рис. 15 Экран входа

В случае отсутствия учетной записи пользователь может перейти к процедуре регистрации. Учтена возможность регистрации аккаунта, как для автора, так и для ученика, интерфейс показан на рисунке 16.

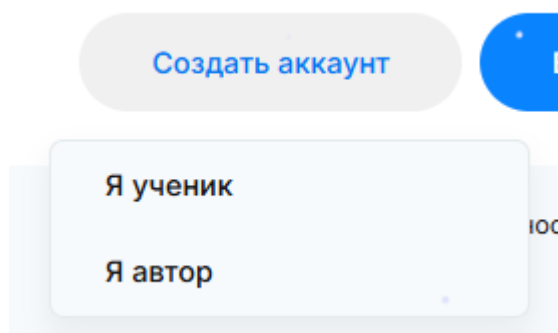


Рис. 16 Меню выбора роли пользователя

На первом этапе регистрации осуществляется ввод персональных данных пользователя, включая фамилию, имя и отчество (при необходимости). Данный шаг показан на рисунке 17. Он позволяет сформировать базовый профиль пользователя в системе и подготовить данные для дальнейшей идентификации.

The image displays a registration form titled "Регистрация ученика". On the right side, there are three input fields: "Фамилия", "Имя", and "Отчество". Below the "Отчество" field, the text "Необязательно" is displayed. At the bottom right of the form, there are two buttons: "Назад" and "Далее >". The background of the form area is decorated with a pattern of small blue dots.

Рис. 17 Первый шаг регистрации ученика

Следующим этапом является подтверждение контактного номера телефона. Пользователь вводит номер телефона, после чего система инициирует отправку одноразового кода подтверждения. Использование механизма подтверждения номера телефона повышает достоверность учетных данных и снижает риск создания фиктивных учетных записей. Данный шаг изображен на рисунке 18.

Номер телефона

Введите номер телефона

Получить код >

Назад Далее >

Рис. 18 Подтверждение номера телефона

После подтверждения контактных данных пользователю предлагается задать пароль для последующей аутентификации. На данном этапе также реализовано получение согласий на обработку персональных данных и условия использования сервиса, что обеспечивает соответствие требованиям законодательства в области защиты информации. Окно создания пароля изображено на рисунке 19.

Пароль

Пароль

Пароль еще раз

☐ С лицензионным договором-офертой согласен

☐ Подтверждаю, что ознакомлен с [Политикой обработки персональных данных](#)

☐ Даю согласие на получение рекламных материалов

Назад Создать аккаунт

Рис. 19 Окно создания пароля

Завершающим этапом регистрации является создание учетной записи и переход пользователя к процедуре входа в систему. После успешной регистрации пользователь может выполнить аутентификацию с использованием указанных ранее учетных данных и получить доступ к функциональным возможностям образовательной платформы. Экран успешного создания аккаунта изображен на 20 рисунке.

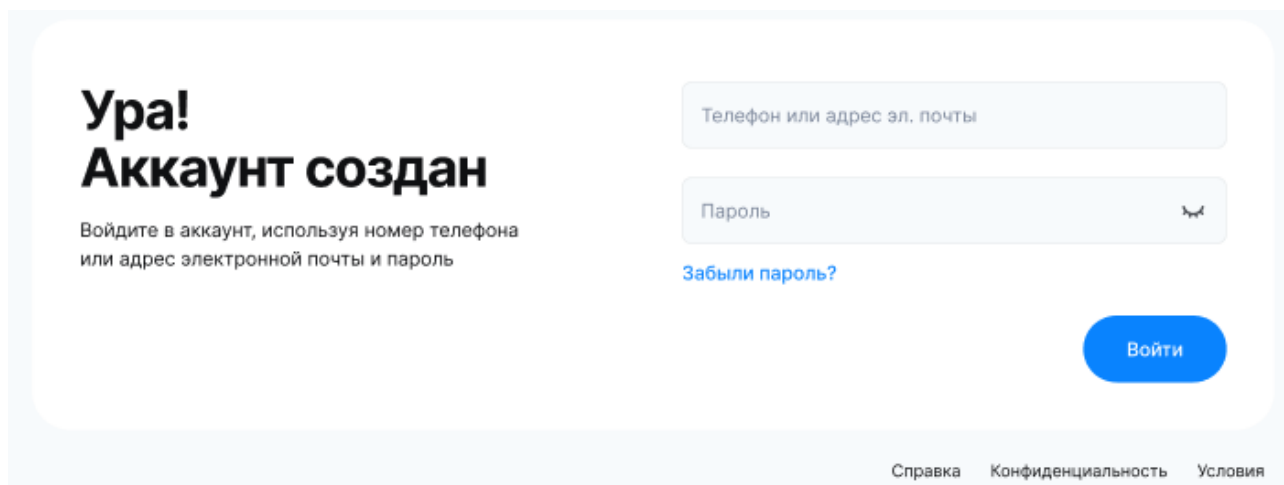


Рис. 20 Экран успешного создания аккаунта

4 Видеокодеки

Одной из ключевых подсистем образовательной платформы является модуль работы с видеоконтентом. Видеоуроки и записи онлайн-эфиров занимают значительный объем хранилища, создают нагрузку на сеть при передаче пользователям и требуют выбора таких параметров кодирования, которые обеспечивают приемлемое качество изображения при разумном размере файлов. Поэтому при проектировании платформы недостаточно только реализовать загрузку и воспроизведение видео. Необходимо определить, какие форматы и параметры кодирования целесообразно использовать для учебных материалов.

В рамках работы было проведено экспериментальное исследование нескольких популярных видеокодеков. Целью исследования являлось сравнение результатов кодирования одного и того же исходного видеоматериала при различных значениях параметра CRF. Анализ выполнялся по нескольким критериям: размер итогового файла, средний битрейт, объективные метрики качества PSNR и SSIM, коэффициент сжатия и время кодирования. Такой набор метрик позволяет оценить не только визуальное качество, но и практическую применимость кодека в составе образовательной платформы.

Для образовательной платформы важен баланс между тремя характеристиками: качеством изображения, скоростью публикации видео и стоимостью хранения. Максимальное качество не всегда является оптимальным решением, поскольку оно приводит к увеличению размера файла и расхода сетевого трафика. С другой стороны, чрезмерное сжатие ухудшает восприятие учебного материала, особенно если в видео присутствуют текст, интерфейсы программ, презентации, схемы или демонстрация кода. Поэтому задача исследования заключается в выборе компромиссного варианта, а не в поиске кодека с одной лучшей характеристикой.

4.1 Сравнение видеокодек

В качестве исходного материала использовался один видеоролик длительностью около 228,7 секунды. Для исключения влияния содержания видео

на результаты все варианты кодировались на основе одного и того же исходного файла. Сравнение проводилось для четырех кодеков: H.264, H.265, VP8 и VP9. Для каждого кодека использовались четыре значения CRF: 18, 23, 28 и 33.

Параметр CRF определяет степень сжатия при кодировании с постоянным качеством. Чем меньше значение CRF, тем выше качество итогового видео и тем больше размер файла. Увеличение CRF приводит к уменьшению размера файла и битрейта, однако одновременно снижает объективные метрики качества. Поэтому анализ зависимости качества, размера и времени кодирования от CRF позволяет определить наиболее рациональные настройки для практического применения.

В исследовании использовались следующие показатели:

- размер итогового файла, характеризующий требуемый объем объектного хранилища;
- средний битрейт, отражающий интенсивность передачи данных при воспроизведении;
- PSNR, показывающий уровень отличия сжатого видео от исходного;
- SSIM, оценивающий структурное сходство изображения с исходным видеоматериалом;
- коэффициент сжатия, показывающий отношение размера исходного файла к размеру результата;
- время кодирования, определяющее затраты на подготовку видео к публикации.

PSNR измеряется в децибелах: чем выше значение, тем меньше различие между исходным и сжатым видео. SSIM принимает значения от 0 до 1, где значение, близкое к 1, означает высокое структурное сходство с исходным изображением. При анализе результатов необходимо учитывать обе метрики, так как PSNR оценивает пиксельную ошибку, а SSIM лучше отражает сохранение визуальной структуры кадра.

Результаты экспериментального кодирования представлены в таблице 2. В таблице приведены значения для каждого кодека и каждого исследуемого

значения CRF. Исходный файл имел размер 98,67 МБ, что использовалось при расчете коэффициента сжатия.

Таблица 2 – Результаты исследования кодеков при различных значениях CRF

Кодек	CRF	Размер (байт)	Размер (читае-мый)	Длительность (сек)	Средний битрейт (кбит/с)	PSNR	SSIM	Кэф. сжатия	Время
-	-	103463719	98.67 MB	228.669	3620.110	-	-	-	-
H.264	18	128866917	122.90 MB	228.669	4508.418	51.324903	0.995914	0,80	42.134
H.264	23	79212231	75.54 MB	228.669	2771.245	48.957715	0.993940	1,31	35.526
H.264	28	50008161	47.69 MB	228.669	1749.539	46.338257	0.990536	2,07	32.769
H.264	33	32396800	30.90 MB	228.669	1133.404	43.592087	0.985028	3,19	29.140
H.265	18	101678592	96.97 MB	228.669	3557.232	50.802732	0.995152	1,02	38.502
H.265	23	54553763	52.03 MB	228.669	1908.567	48.486837	0.992839	1,90	36.340
H.265	28	30457658	29.05 MB	228.669	1065.563	46.157915	0.989631	3,40	26.894
H.265	33	18347279	17.50 MB	228.669	641.881	43.662501	0.984834	5,64	18.179
VP8	18	141606746	135.05 MB	228.653	4954.468	47.612439	0.991913	0,73	70.198
VP8	23	120391870	114.81 MB	228.653	4212.212	47.150637	0.991178	0,86	63.285
VP8	28	98333989	93.78 MB	228.653	3440.462	46.507647	0.989996	1,05	60.689
VP8	33	76135302	72.61 MB	228.653	2663.785	45.592378	0.988340	1,36	57.234
VP9	18	111049328	105.90 MB	228.653	3885.340	50.649724	0.994733	0,93	349.423
VP9	23	86484809	82.48 MB	228.653	3025.888	49.801680	0.993915	1,20	324.919
VP9	28	61733833	58.87 MB	228.653	2159.913	48.646723	0.992650	1,68	299.939
VP9	33	44228949	42.18 MB	228.653	1547.461	47.446265	0.991126	2,34	268.506

По полученным данным видно, что уменьшение размера файла при увеличении CRF наблюдается у всех исследованных кодеков. При этом характер снижения качества и время обработки различаются достаточно существенно. Это подтверждает необходимость выбора кодека не только по одному показателю, например по размеру файла, а по совокупности характеристик.

На рисунке 21 представлена зависимость PSNR от значения CRF для исследованных кодеков. Общая закономерность ожидаема: при увеличении CRF качество видео снижается, что выражается в уменьшении PSNR. Однако снижение происходит неодинаково для разных кодеков.

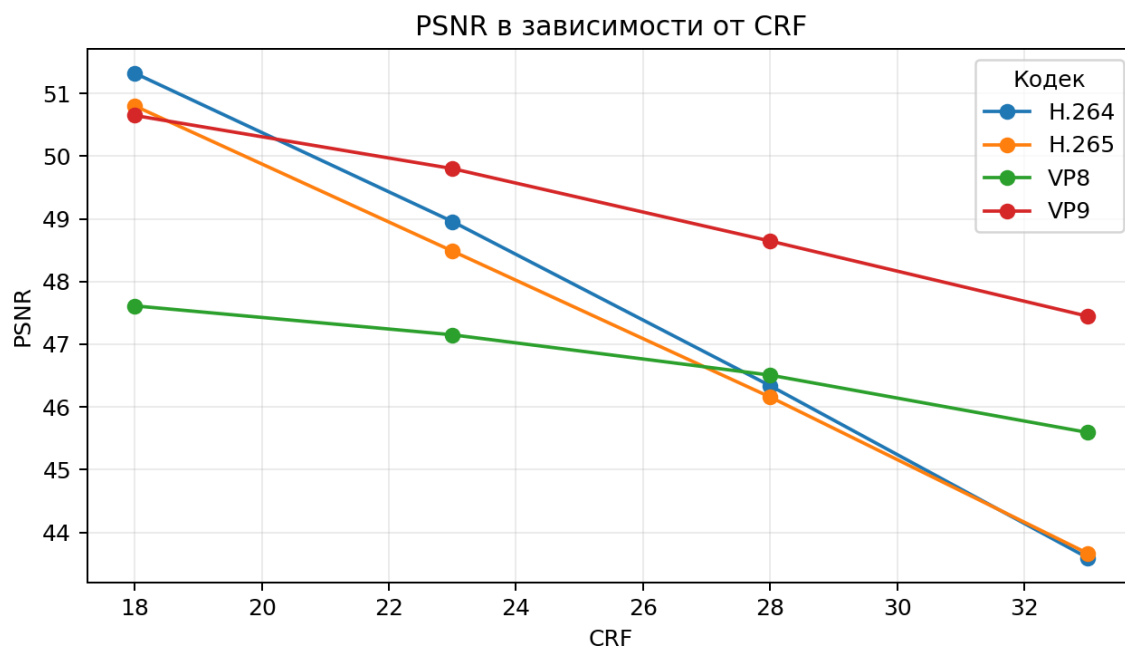


Рис. 21 – Зависимость PSNR от CRF для исследованных кодеков

На рисунке 22 представлена зависимость времени кодирования от CRF. Время кодирования является критически важным параметром для образовательной платформы. Если автор курса загружает видеоурок или запись эфира, материал должен быть подготовлен к публикации за приемлемое время. Слишком медленная обработка ухудшает пользовательский опыт и усложняет эксплуатацию платформы.

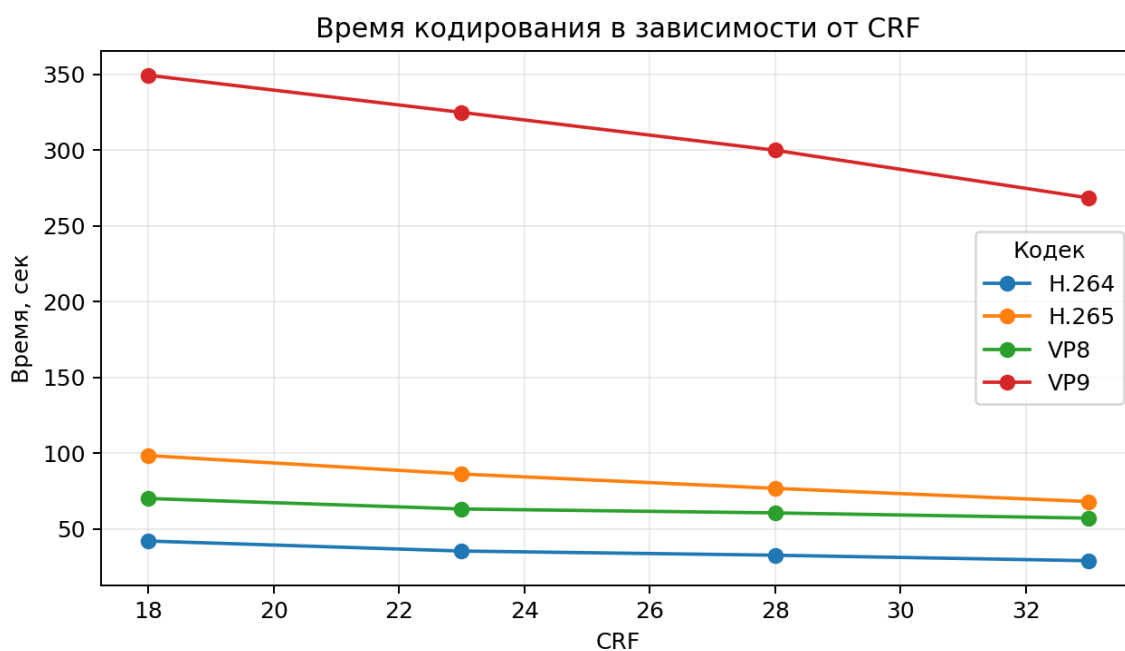


Рис. 22 – Зависимость времени кодирования от CRF

На рисунке 23 представлена зависимость размера файла от CRF. Размер файла напрямую влияет на стоимость хранения видеоматериалов и объем передаваемого трафика. Для образовательной платформы это особенно важно, поскольку видеоконтент может составлять основную часть всех хранимых данных.

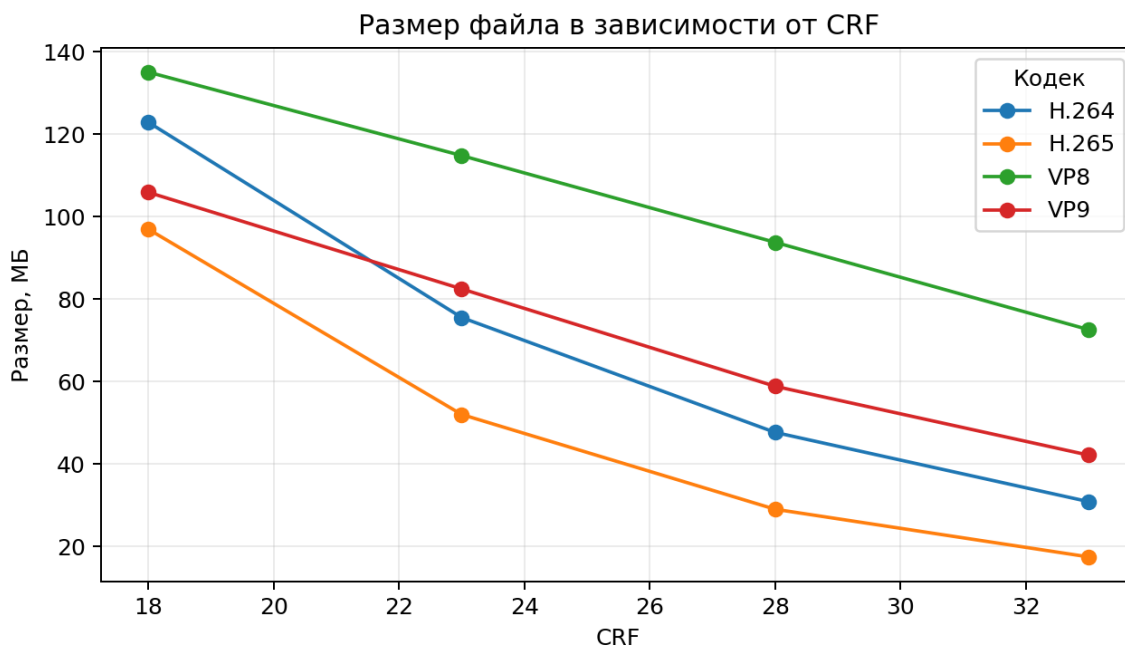


Рис. 23 – Зависимость размера файла от CRF

Таким образом, для образовательной платформы рационально использовать комбинированный подход. Для быстрой публикации и максимальной совместимости следует использовать H.264 с CRF 23. Для уменьшения объема хранения можно дополнительно применять H.265 с CRF 28 (Например, для видео в высоком разрешении). VP9 может использоваться как вариант фоновой обработки при необходимости получить более высокое качество при сильном сжатии, если допустимы повышенные временные затраты. Такой подход позволяет сбалансировать качество видеоматериалов, скорость обработки, стоимость хранения и удобство воспроизведения на стороне пользователя.

5 Структура курса

Курс является одной из ключевых сущностей разработанной образовательной платформы. Именно через курс в системе организуется представление учебного контента, взаимодействие автора с учениками, управление доступом к материалам, настройка модулей и уроков, а также параметры публикации и защиты видеоконтента. В рамках разработанного решения курс рассматривается не как единичная запись в базе данных, а как сложный объект, объединяющий набор взаимосвязанных подсистем.

С точки зрения логики платформы курс включает в себя общую информацию, описание ожидаемых результатов обучения, параметры связи с автором, механизмы коммуникации между участниками, структуру уроков и модулей, средства защиты видеоконтента, а также настройки публикации и запуска. Такой подход позволяет автору пошагово заполнить все необходимые сведения и подготовить курс к дальнейшему размещению в системе.

Интерфейс создания курса реализован в виде последовательного мастера настройки, где каждый шаг отвечает за отдельный аспект курса. Это упрощает работу пользователя и делает процесс подготовки курса более понятным и управляемым. Последовательность шагов включает: заполнение общей информации о курсе, указание контактов для связи с учениками до и после покупки, настройку чатов курса, добавление уроков, задание порядка модулей и уроков, подключение защиты видео и задание параметров запуска курса.

5.1 Общая информация о курсе

На первом этапе создается базовая информационная структура курса. Автор указывает категорию курса, его название, краткое и подробное описание, а также формулирует перечень навыков или результатов, которые ученик получит после прохождения обучения. Дополнительно на данном этапе задаются стоимость и уровень сложности курса. Также автор загружает главное изображение курса и материалы для обзорного раздела, которые используются в интерфейсе представления курса пользователям.

Такой набор полей необходим для формирования первичного описания курса и его позиционирования внутри платформы. Категория позволяет включить курс в соответствующий тематический раздел. Название и описание используются для отображения в каталогах и карточке курса. Перечень навыков помогает пользователю понять, каких практических результатов он сможет достичь. Стоимость и сложность выступают как важные параметры при выборе курса. Изображения и обзорные материалы повышают наглядность и улучшают восприятие контента. Интерфейс представлен на рисунке 24.

Рис. 24 Интерфейс заполнения общей информации о курсе

5.2 Контакты до и после покупки курса

Следующими этапами являются настройки средств связи, доступных пользователю до и после приобретения курса, соответственно. На данном шаге автор может указать контактные данные, по которым потенциальный ученик сможет задать вопросы, уточнить содержание программы или получить дополнительную консультацию перед и после покупки. Это может быть номер телефона, ссылка на мессенджер, адрес электронной почты или иной канал связи.

Наличие такой функциональности имеет важное практическое значение. Общение с автором курса или администраторами способствует росту доверия со стороны пользователей, позволяет снять сомнения и в целом положительно

влияет на конверсию. Интерфейсы контактов до и после покупки указаны на рисунке 25 и 26, соответственно.

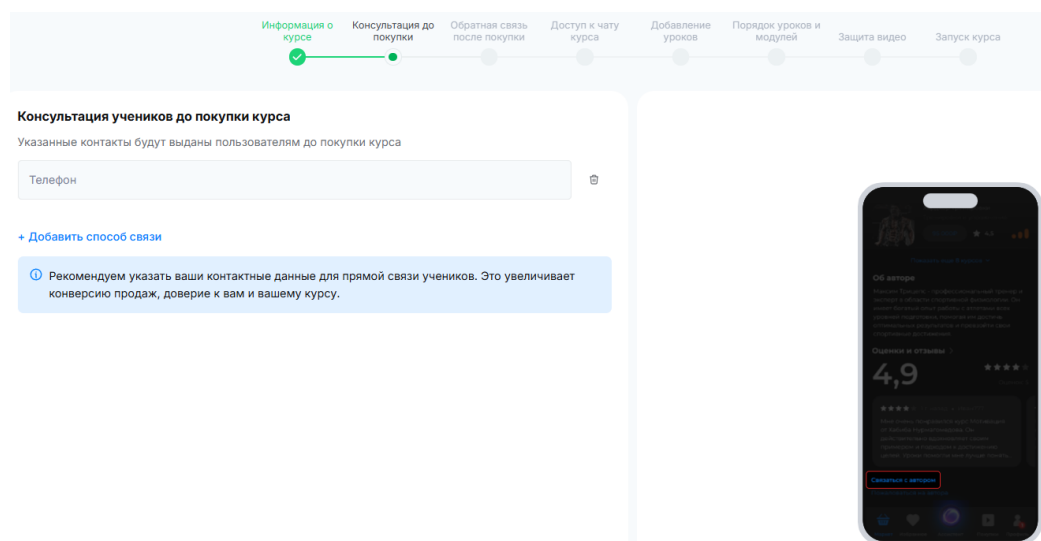


Рис. 25 Настройка контактов до покупки

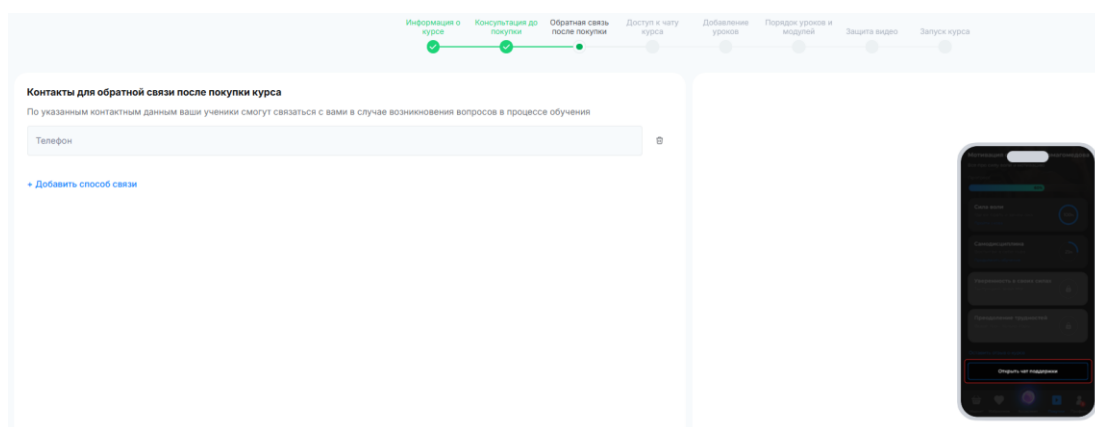


Рис. 26 Настройка контактов после покупки

5.3 Чаты курса

Важным элементом структуры курса является коммуникационная подсистема. В разработанной платформе для каждого курса может быть организовано взаимодействие участников через чаты. На этапе настройки автор может задать единые чаты, доступные всем ученикам, а также региональные чаты, предназначенные для пользователей из определённых часовых поясов. Интерфейс этапа представлен на рисунке 27

Разделение чатов на общие и региональные позволяет повысить удобство общения. Общие чаты используются для обсуждения содержательных вопросов, связанных с прохождением курса, обмена мнениями и публикации общих

объявлений. Региональные чаты позволяют учитывать географические и временные особенности аудитории. Это особенно актуально для курсов с большой распределенной аудиторией, где пользователи находятся в разных странах и часовых поясах. Интерфейс региональных чатов представлен на рисунке 28.

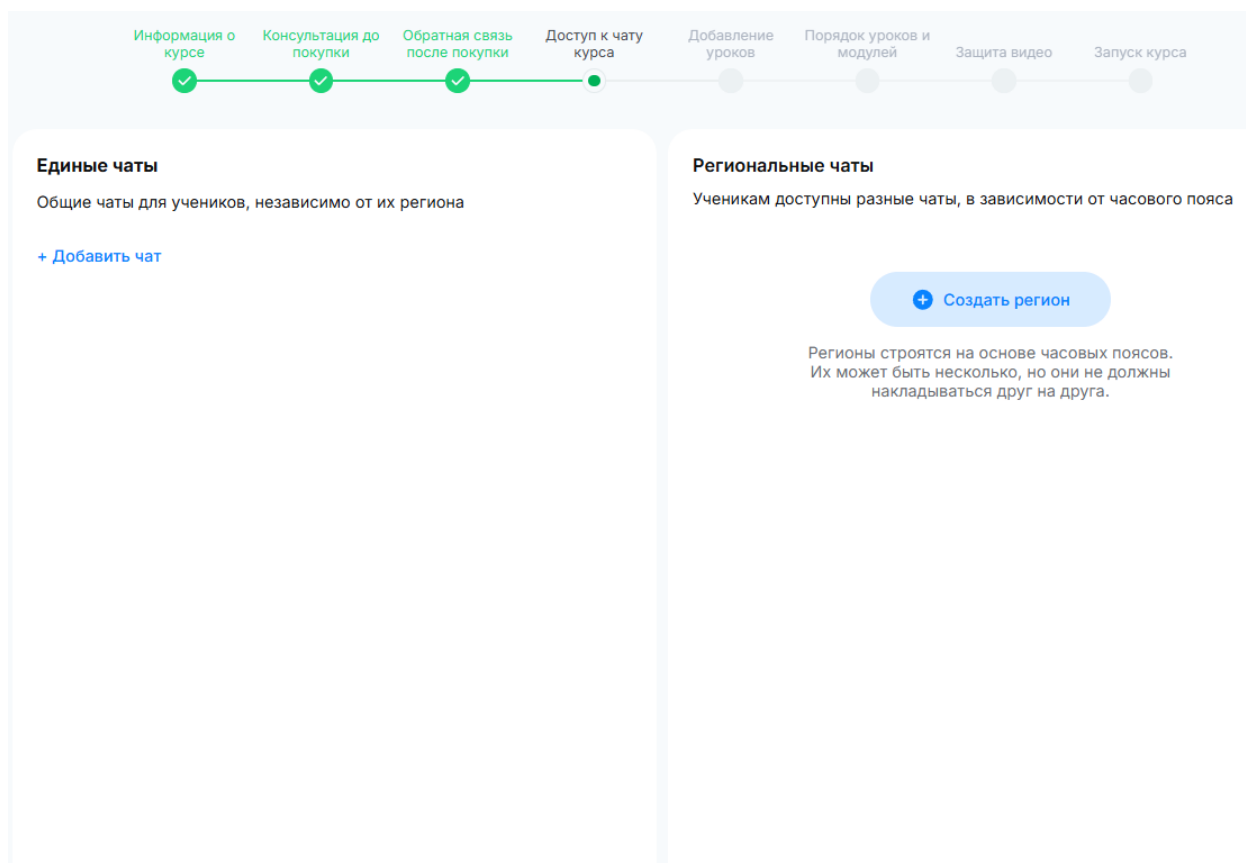


Рис. 27 Настройка чатов

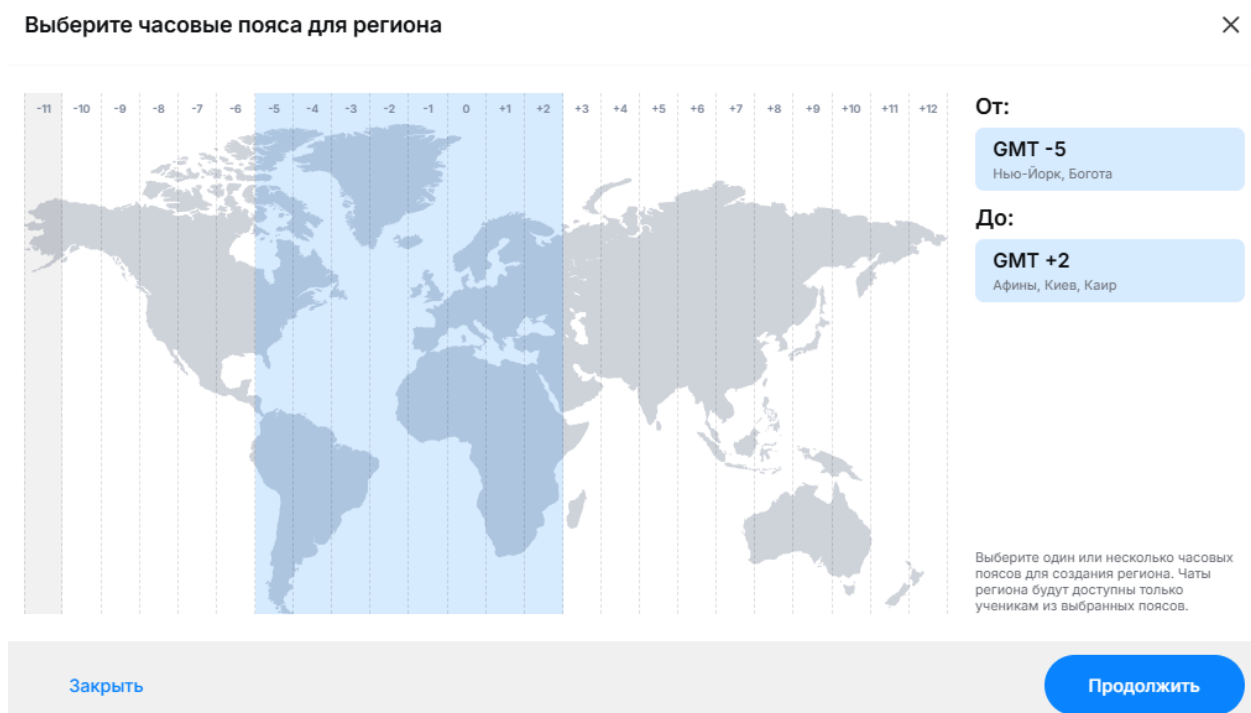


Рис. 28 Настройка региональных чатов

5.4 Уроки курса

После настройки средств коммуникации автор переходит к созданию содержательной части курса. На данном этапе осуществляется добавление уроков. Для каждого урока задаются название, краткое и подробное описание, обложка, видеоматериал и дополнительные учебные материалы. Такая структура урока позволяет объединить основной видеоконтент и сопроводительные файлы в единый учебный блок.

Урок выступает базовой единицей представления контента в образовательной платформе. С точки зрения предметной области именно урок содержит конкретный фрагмент учебной информации, который должен быть изучен пользователем. Возможность прикрепления материалов к уроку расширяет функциональность платформы и позволяет использовать не только видеоформат, но и дополнительные документы, презентации, инструкции и иные ресурсы. Интерфейс урока показан на 29 рисунке.

Рис. 29 Настройка урока

5.5 Модули курса

Для упорядочивания большого объема учебного материала курс разбивается на модули. Каждый модуль может включать несколько уроков, объединенных общей темой или логикой прохождения. На соответствующем этапе настройки автор формирует модульную структуру курса, добавляет новые модули, распределяет по ним уроки и определяет их последовательность.

Подобная организация позволяет выстроить учебный процесс в логически завершенном иерархическом виде. Модуль представляет собой более крупную единицу структуры, чем урок, и может использоваться для тематической группировки, выделения этапов прохождения или формирования отдельных блоков программы. Возможность изменения порядка модулей и уроков повышает гибкость управления курсом и позволяет автору адаптировать структуру под конкретную образовательную методику. На 30 рисунке изображен экран настройки модуля.

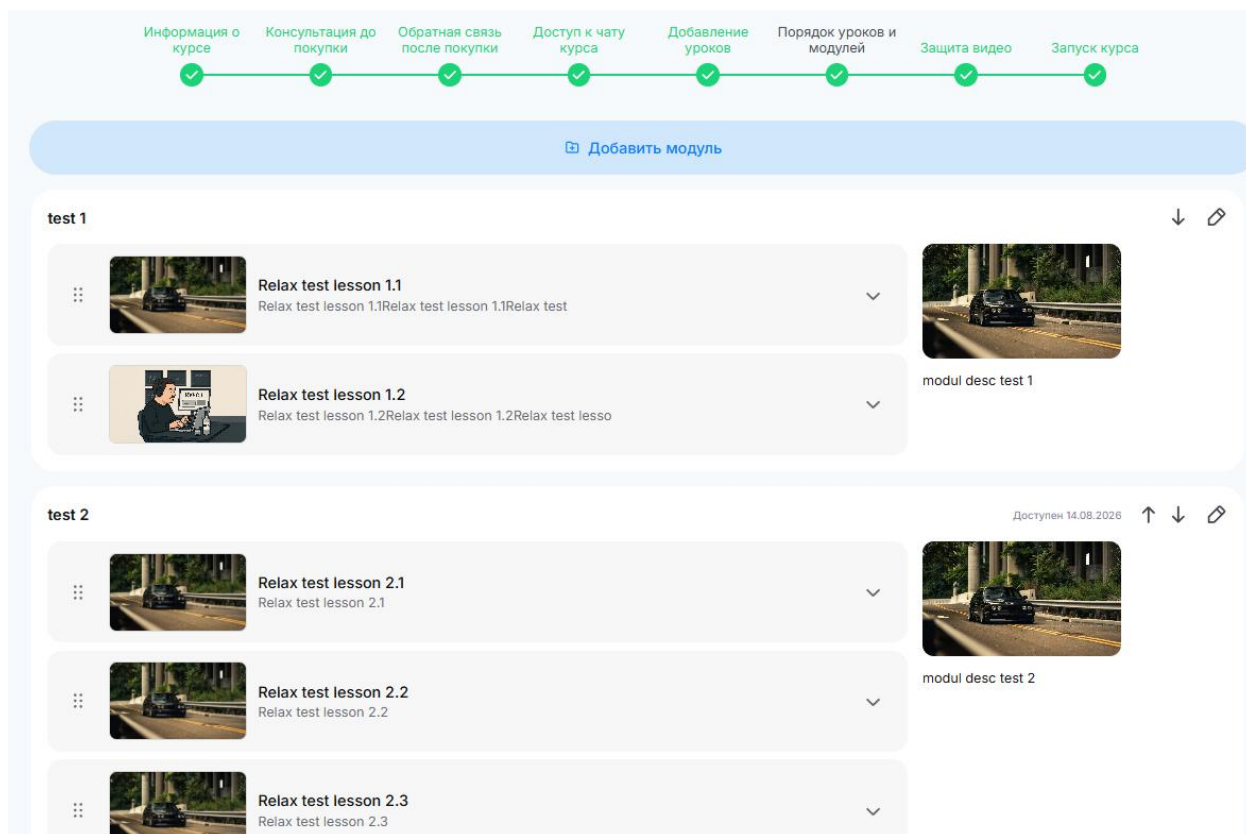


Рис. 30 Настройка модуля

5.6 Защита видеоконтента

Поскольку основным форматом представления учебного материала в платформе является видео, важной частью структуры курса является механизм защиты видеоконтента. Для этого в системе реализована возможность подключения визуального маркирования воспроизводимого видео. Суть метода заключается в динамическом наложении уникального визуального идентификатора пользователя на изображение в процессе просмотра.

Такой подход позволяет повысить защищённость материалов от несанкционированного копирования и распространения. Если пользователь попытается записать экран или распространить видеоконтент, наложенная метка позволит установить источник утечки. Включение данного механизма в настройки курса позволяет автору самостоятельно определять, требуется ли дополнительная защита для конкретного образовательного продукта. На рисунке 31 изображен экран настройки защиты видеоконтента.

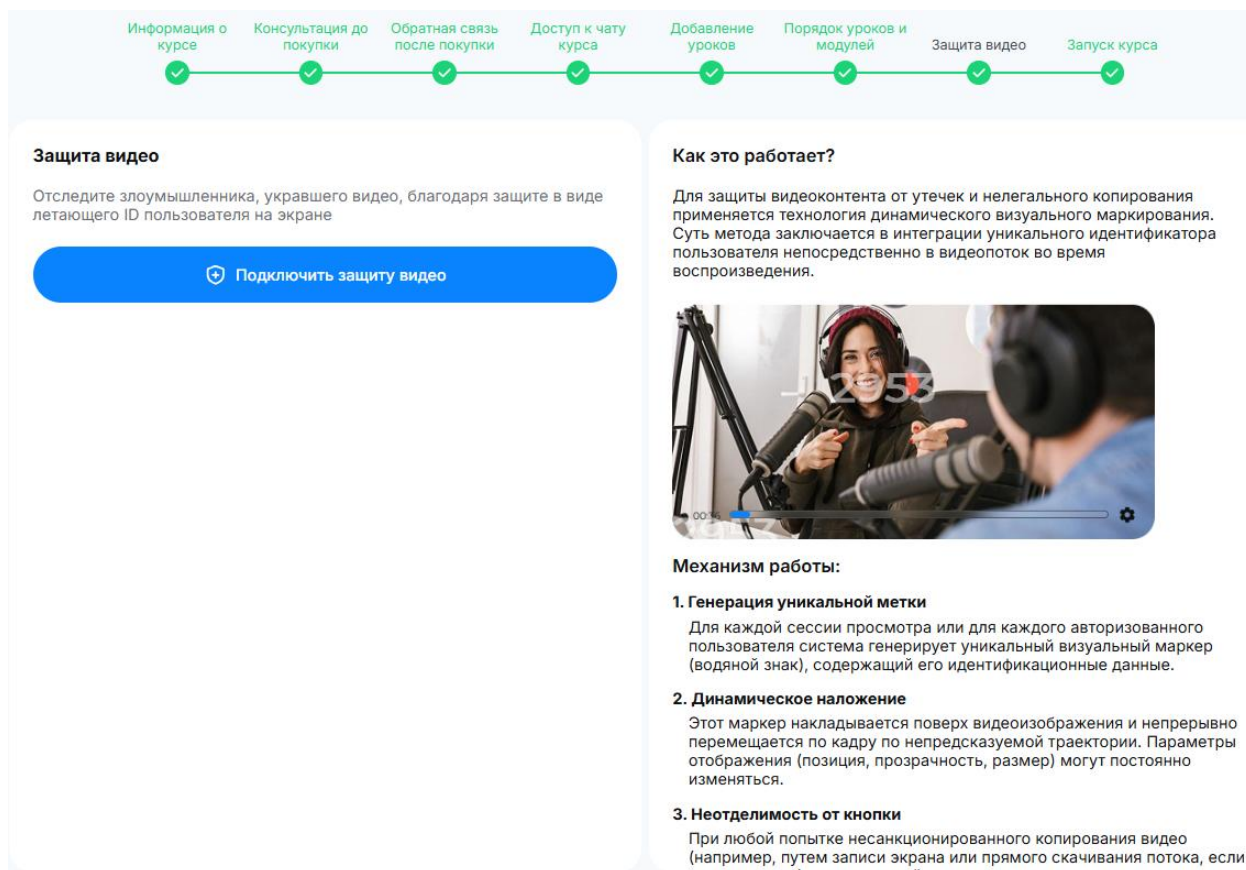


Рис. 31 Настройка защиты видеоконтента

5.7 Запуск курса

Заключительным этапом является настройка параметров публикации и запуска курса. Автор может выбрать немедленный запуск после публикации или назначить конкретные дату и время начала. Дополнительно указывается часовой пояс, что особенно важно для платформы, ориентированной на пользователей из разных регионов. Также система позволяет задать продолжительность доступности курса: курс может быть доступен бессрочно или в течение ограниченного периода после запуска.

Данный этап завершает формирование структуры курса как полноценного объекта образовательной системы. Если предыдущие этапы определяли содержание, коммуникацию и организацию материала, то на последнем шаге задаются правила начала и продолжительности доступа. Это необходимо для реализации различных образовательных сценариев: моментального старта, запуска по расписанию, временного доступа, потокового обучения или

ограниченных по времени образовательных программ. Интерфейс данного этапа изображен на рисунке 32.

Рис. 32 Настройка запуска курса

6 Структура эфира

Эфир является важной сущностью разработанной образовательной платформы и предназначен для организации синхронного взаимодействия автора курса со зрителями в режиме реального времени. В отличие от заранее записанных видеоуроков, эфир позволяет проводить трансляции, оперативно взаимодействовать с аудиторией, отвечать на вопросы участников и использовать дополнительные механизмы вовлечения пользователей. В рамках платформы эфир рассматривается как отдельный объект, связанный с курсом, параметрами доступа, подсистемой чатов и настройками посттрансляционного поведения.

С точки зрения предметной области структура эфира включает несколько логических блоков: общую информацию, параметры доступа к трансляции и настройки поведения после завершения эфира. Такое деление позволяет последовательно настроить все ключевые характеристики эфира и подготовить его к публикации. Пошаговый интерфейс облегчает процесс создания трансляции и снижает вероятность пропуска важных параметров.

Последовательность настройки эфира в системе реализована в виде мастера, включающего три этапа: ввод общей информации о трансляции, определение правил доступа и настройку сценариев после завершения эфира. Каждый этап отвечает за собственный набор параметров и вместе формирует полную конфигурацию эфира как самостоятельной сущности образовательной платформы.

6.1 Общая информация о эфире

На первом этапе создается базовая информационная структура эфира. Автор указывает название и описание эфира, задает дату и время начала трансляции, выбирает часовой пояс, а также может настроить рекламную кнопку с заголовком и ссылкой. Дополнительно загружается обложка эфира, которая используется как основной визуальный элемент при отображении трансляции на платформе, а также может выступать заставкой до начала и после завершения эфира.

Наличие данных параметров необходимо для корректного представления эфира пользователям. Название и описание позволяют донести до аудитории содержание предстоящего мероприятия. Указание времени запуска и часового пояса обеспечивает корректную синхронизацию с пользовательским интерфейсом и позволяет учитывать распределённую аудиторию. Обложка повышает наглядность представления трансляции, а рекламная кнопка может использоваться для размещения дополнительной ссылки, например на сопутствующий материал, форму регистрации или внешний ресурс. Интерфейс ввода общей информации о эфире представлен на 33 рисунке.

Общая информация Доступы После эфира

Общая информация

Название эфира 0 из 40

Описание эфира 0 из 5000
[Развернуть](#)

Начало эфира

Укажите дату запуска ЧЧ:ММ

(UTC +03:00) Москва, Санкт-Петербург, Волгоград ▼

Рекламная кнопка
Кнопка СТА, которую можно показывать и скрывать в любой момент эфира.

Заголовок кнопки 0 из 40

Ссылка

Обложка эфира
Рекомендуемый размер: 1920x1080px

Загрузить

Загруженное изображение используется как обложка, а также как заставка перед эфиром и после него.

Рис. 33 Интерфейс заполнения общей информации о эфире

6.2 Параметры доступа эфира

Второй этап посвящен определению правил доступа к трансляции. В разработанной платформе предусмотрено несколько режимов подключения зрителей к эфиру. Автор может выбрать открытый доступ, при котором подключение к трансляции доступно любому пользователю, режим с обязательным вводом имени и телефона для сбора контактов потенциальных клиентов, режим доступа только для авторизованных пользователей, а также

режим, при котором трансляция доступна исключительно ученикам, приобретшим соответствующий курс.

Такой набор вариантов обеспечивает гибкость использования эфиров в различных сценариях. Открытый доступ подходит для рекламных и ознакомительных трансляций. Сбор имени и телефона может использоваться в маркетинговых целях для последующей коммуникации с аудиторией. Доступ только для авторизованных пользователей позволяет ограничить просмотр рамками зарегистрированной аудитории платформы. Доступ только ученикам курса обеспечивает использование эфира как части закрытого образовательного процесса.

На данном же этапе производится настройка параметров чата. Автор может ограничить возможность отправки сообщений только собой либо разрешить общение зрителям. При этом система допускает выбор круга пользователей, которым разрешено писать в чат: всем зрителям, только авторизованным в системе пользователям или пользователям, прошедшим авторизацию через Telegram. Дополнительно доступны ограничения по частоте отправки сообщений и вспомогательные параметры, например запрет или разрешение на отправку ссылок. Эти механизмы позволяют повысить управляемость общения и снизить риск спама во время трансляции. Интерфейс данного этапа изображен на рисунке 34.

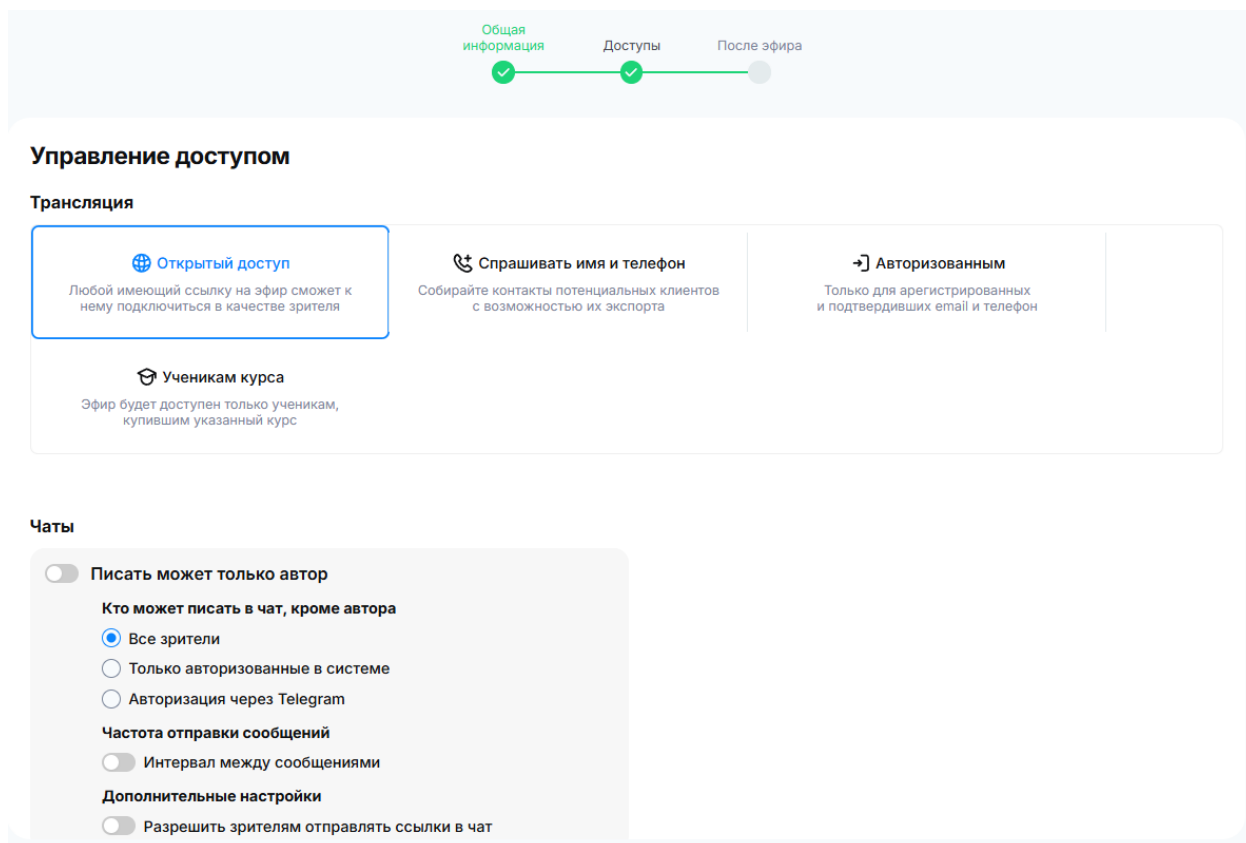


Рис. 34 Интерфейс настройки доступов эфира

6.3 Настройка поведения после завершения эфира

Третий этап описывает поведение системы после окончания трансляции. На данном шаге автор может выбрать сохранение записи эфира для дальнейшего использования, а также настроить автоматическую переадресацию зрителей после завершения эфира на заданную страницу. Сохранение записи позволяет повторно использовать контент в составе курса или как самостоятельный видеоматериал. Переадресация после эфира может применяться для перехода пользователя на страницу курса, форму обратной связи, страницу оплаты либо иной целевой раздел системы.

Настройка действий после завершения трансляции имеет важное значение как с образовательной, так и с организационной точки зрения. Запись эфира расширяет жизненный цикл созданного контента и позволяет использовать его повторно. Автоматическая переадресация упрощает построение пользовательских сценариев и помогает направить аудиторию к следующему этапу взаимодействия с платформой. Интерфейс данного этапа изображен на рисунке 35.

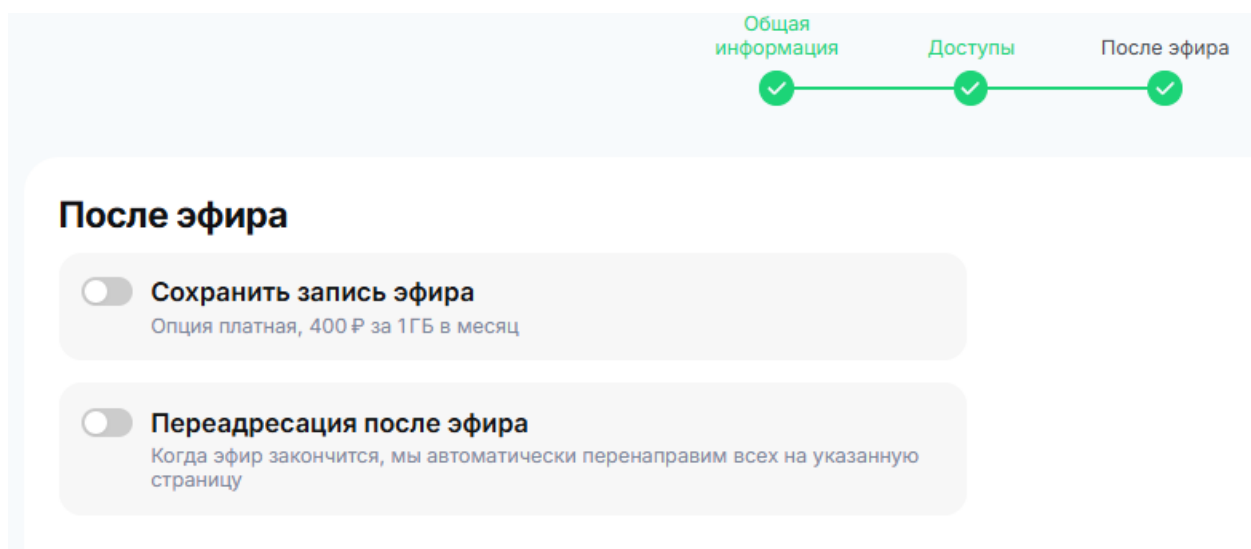


Рис. 35 Интерфейс настройки параметров после завершения эфира

7 Описание видеоплеера

Видеоплеер является одним из ключевых пользовательских компонентов образовательной платформы, поскольку именно через него осуществляется просмотр основного учебного контента. Для большинства онлайн-курсов видеоматериалы выступают центральным способом передачи информации, поэтому качество реализации видеоплеера напрямую влияет на удобство обучения, вовлеченность пользователей и общее восприятие платформы.

В рамках разработанной системы видеоплеер используется для воспроизведения видеоуроков, записей эфиров и иных учебных видеоматериалов, связанных с курсами. Он должен обеспечивать стабильное воспроизведение, удобное управление просмотром, поддержку выбора качества, возможность изменения скорости воспроизведения, а также корректную работу с защищённым контентом. В отличие от обычного встроенного HTML-видео, видеоплеер образовательной платформы является частью общей архитектуры доступа к учебным материалам и взаимодействует с backend-сервисами, системой авторизации и хранилищем видеоконтента. Интерфейс видеоплеера изображен на рисунке 36



Рис. 36 Интерфейс видеоплеера

8 Запуск платформы

Завершающим этапом разработки образовательной платформы является развертывание всех необходимых сервисов в единой инфраструктурной среде. Поскольку система построена как набор отдельных backend-сервисов и вспомогательных компонентов, для ее запуска целесообразно использовать Kubernetes. Такой подход позволяет централизованно управлять контейнерами, конфигурациями, сетевым взаимодействием, переменными окружения, перезапуском сервисов и масштабированием отдельных компонентов.

Для запуска платформы необходимо подготовить набор конфигурационных файлов Kubernetes. В их состав входят манифесты Deployment, описывающие запуск отдельных сервисов, Service для организации сетевого доступа между компонентами, ConfigMap для хранения параметров конфигурации и Secret для хранения чувствительных данных, таких как пароли, ключи доступа и строки подключения. Дополнительно для сервисов, использующих постоянное хранение данных, могут применяться PersistentVolume и PersistentVolumeClaim.

В рамках платформы в Kubernetes разворачиваются основные сервисы системы, а также вспомогательные обработчики. Отдельно запускаются инфраструктурные компоненты: PostgreSQL, Redis, Kafka, Centrifugo, Ory Hydra, Ory Keto, сервисы мониторинга и экспортеры метрик. Такое разделение позволяет независимо обновлять и масштабировать отдельные части платформы без остановки всей системы.

Использование Kubernetes также упрощает эксплуатацию платформы. В случае сбоя отдельного контейнера Kubernetes может автоматически перезапустить под, что повышает отказоустойчивость системы. Кроме того, через механизм ReplicaSet обеспечивается контроль количества запущенных экземпляров сервисов. Это особенно важно для платформы с большим количеством подсистем, где ручной запуск каждого компонента был бы неудобным и ненадежным решением.

После подготовки конфигураций и применения манифестов все компоненты платформы были успешно запущены в Kubernetes-кластере. На рисунке 37 представлен результат разворачивания.

☐	Name	 Namespace	Cont...	CPU	Memor	Restart	Controlled B
☐	auth-578ddd65d4	default	 	0.000	8.4MiB	0	ReplicaSet
☐	centrifugo-5b96c	default		0.001	36.9MiB	0	ReplicaSet
☐	chat-7dd99d4d6d	default	 	0.001	9.6MiB	0	ReplicaSet
☐	course-5868944fc	default	 	0.001	13.6MiB	0	ReplicaSet
☐	course-activating-	default		0.000	5.5MiB	0	ReplicaSet
☐	course-monitor-d	default		0.001	11.4MiB	0	ReplicaSet
☐	files-7fc746d5c9-7	default	 	0.006	15.6MiB	0	ReplicaSet
☐	fluent-bit-6psfk	default		0.000	4.2MiB	0	DaemonSet
☐	fluent-bit-pvgsn	default		0.000	4.3MiB	0	DaemonSet
☐	frontend-6d5fdfb	default		0.000	7.5MiB	0	ReplicaSet
☐	gateway-55fd5d7l	default		0.000	15.8MiB	0	ReplicaSet
☐	kafka-exporter-7c	default		0.001	15.8MiB	0	ReplicaSet
☐	kafka-ui-dcbf5d74	default		0.002	394.2MiB	0	ReplicaSet
☐	keto-5fcfdb8885-t	default	 	0.000	19.3MiB	0	ReplicaSet
☐	postgres-exporter	default		0.001	13.5MiB	0	ReplicaSet
☐	promo-58d96d97	default		0.000	6.4MiB	0	ReplicaSet
☐	redis-exporter-9cl	default		0.000	8.6MiB	0	ReplicaSet
☐	role-service-6f944	default		0.000	5.7MiB	0	ReplicaSet
☐	stream-starting-n	default		0.000	13.4MiB	0	ReplicaSet
☐	telegram-7487f86	default	 	0.000	6.3MiB	0	ReplicaSet
☐	telegram-consum	default		0.002	7.6MiB	0	ReplicaSet
☐	telegram-webhoo	default		0.000	6.9MiB	0	ReplicaSet
☐	user-5474476ffd-i	default	 	0.000	55.8MiB	0	ReplicaSet
☐	video-consumer-7	default		0.001	19.0MiB	0	ReplicaSet
☐	video-progress-64	default	 	0.001	10.8MiB	2	ReplicaSet
☐	video-progress-cc	default		0.002	11.0MiB	0	ReplicaSet
☐	video-server-5f44	default	 	0.001	23.0MiB	0	ReplicaSet

Рис. 37 Все поды проекта, запущенные в Kubernetes

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель — исследована и разработана образовательная платформа с учебными курсами, обеспечивающая управление пользователями, курсами, уроками, медиаконтентом и доступом к ресурсам системы.

В рамках работы была спроектирована и описана архитектура платформы, построенная на основе распределенного набора сервисов. Серверная часть реализована с использованием языка программирования Golang. Для хранения структурированных данных применяется PostgreSQL, для временных данных и кэширования — Redis, для асинхронного событийного взаимодействия — Kafka, для хранения файлов и видеоматериалов — S3-совместимое объектное хранилище MinIO, а для доставки сообщений и уведомлений в режиме реального времени — Centrifugo. Такой набор технологий позволил построить расширяемую архитектуру, пригодную для дальнейшего масштабирования и развития платформы.

В процессе работы были исследованы подходы к аутентификации и авторизации пользователей в современных веб-системах. Рассмотрены механизмы единого входа, протоколы OAuth 2.0 и OpenID Connect, а также особенности применения JWT-токенов. На основе проведенного анализа для реализации авторизации был выбран Ory Hydra, выполняющий роль authorization server и обеспечивающий поддержку современных стандартов авторизации. Практическое применение Ory Hydra позволило вынести критически важную логику выдачи и обработки токенов в отдельный специализированный компонент.

Отдельное внимание было уделено вопросам разграничения доступа. В работе были рассмотрены основные модели управления доступом, включая DAC, MAC, ACL, RBAC, ABAC и ReBAC. Проведенный анализ показал, что для образовательной платформы с курсами, уроками, эфирами, чатами и материалами наиболее подходящей является модель доступа на основе отношений. В связи с этим для реализации гибкой проверки прав был выбран

Ory Keto. Его использование позволяет описывать доступ не только через роли пользователей, но и через отношения между пользователями и конкретными объектами системы, например курсами, уроками, материалами и эфирами.

Была разработана структура ключевых сущностей образовательной платформы. Описана структура курса, включающая общую информацию, контакты до и после покупки, чаты, уроки, модули, защиту видеоконтента и параметры запуска. Также была рассмотрена структура онлайн-эфира, включающая общую информацию, параметры доступа, настройки чата и действия после завершения трансляции. Данные разделы показывают, что курс и эфир в системе являются не простыми записями, а комплексными объектами, связанными с несколькими функциональными подсистемами платформы.

В рамках практической реализации были описаны основные пользовательские сценарии: регистрация и авторизация пользователя, создание и настройка курса, добавление уроков и модулей, настройка чатов, загрузка и воспроизведение видеоконтента, проведение онлайн-эфиров и работа с видеоплеером. Реализованный видеоплеер обеспечивает просмотр учебных материалов, выбор качества, управление скоростью воспроизведения и взаимодействие с системой проверки доступа к контенту.

Также в работе было проведено исследование видеокодеков, применимых для хранения и доставки учебного видеоконтента. Были рассмотрены кодеки H.264, H.265, VP8 и VP9 при различных значениях параметра CRF. Сравнение выполнялось по размеру итогового файла, времени кодирования, среднему битрейту, PSNR, SSIM и коэффициенту сжатия. Полученные результаты показали, что выбор кодека должен выполняться с учетом компромисса между качеством изображения, временем обработки и объемом хранения. Для образовательной платформы наиболее рациональным является комбинированный подход: использование H.264 для быстрой публикации и высокой совместимости, а также применение более эффективных кодеков для сценариев, где важнее экономия дискового пространства и трафика.

Таким образом, все поставленные в работе задачи были выполнены. Был проведен анализ современных образовательных платформ и их требований, исследованы подходы к аутентификации, авторизации и управлению доступом, разработана модель доступа для образовательной платформы, реализованы механизмы авторизации и разграничения доступа с использованием Ory Hydra и Ory Keto, спроектирована архитектура системы, описаны основные сущности платформы и реализованы ключевые подсистемы, связанные с курсами, видео, онлайн-эфирами, чатами и уведомлениями.

Практическая значимость выполненной работы заключается в том, что разработанная архитектура и программная реализация могут быть использованы как основа для дальнейшего развития образовательной платформы. Предложенное решение объединяет управление учебным контентом, безопасную авторизацию, гибкое разграничение доступа, работу с видеоматериалами, проведение онлайн-эфиров и real-time взаимодействие пользователей.

В дальнейшем развитие платформы может быть связано с добавлением системы домашних заданий и тестирования, расширением аналитики прохождения курсов, внедрением рекомендаций учебных материалов, улучшением механизмов защиты видеоконтента, развитием мобильного приложения и повышением отказоустойчивости отдельных сервисов. Кроме того, перспективным направлением является расширение мониторинга работы системы и оптимизация обработки видеоматериалов при росте количества пользователей и объема образовательного контента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. RFC 6749. «The OAuth 2.0 Authorization Framework» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 05.10.2025 - 06.02.2026).
2. RFC 6750. «The OAuth 2.0 Authorization Framework: Bearer Token Usage» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc6750> (дата обращения: 05.10.2025 - 06.02.2026).
3. RFC 7636. «Proof Key for Code Exchange by OAuth Public Clients (PKCE)» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc7636> (дата обращения: 05.10.2025 - 06.02.2026).
4. RFC 8252. «OAuth 2.0 for Native Apps» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/rfc8252/> (дата обращения: 05.10.2025 - 06.02.2026).
5. RFC 7009. «OAuth 2.0 Token Revocation» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc7009> (дата обращения: 05.10.2025 - 06.02.2026).
6. OpenID Connect Core 1.0 (Errata Set 2). «OpenID Connect Core 1.0» [Электронный ресурс] – URL: https://openid.net/specs/openid-connect-core-1_0.html (дата обращения: 05.10.2025 - 06.02.2026).
7. OpenID Connect Discovery 1.0 (Errata Set 2). «OpenID Connect Discovery 1.0» [Электронный ресурс] – URL: https://openid.net/specs/openid-connect-discovery-1_0.html (дата обращения: 05.10.2025 - 06.02.2026).
8. RFC 7519. «JSON Web Token (JWT)» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/rfc7519/> (дата обращения: 05.10.2025 - 06.02.2026).
9. RFC 7515. «JSON Web Signature (JWS)» [Электронный ресурс] – URL: <https://datatracker.ietf.org/doc/html/rfc7515> (дата обращения: 05.10.2025 - 06.02.2026).
10. OWASP. «Application Security Verification Standard (ASVS)» [Электронный ресурс] – URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата обращения: 05.10.2025 - 06.02.2026).

11. OWASP Cheat Sheet Series. «JSON Web Token for Java Cheat Sheet» [Электронный ресурс] – URL: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html (дата обращения: 05.10.2025 - 06.02.2026).
12. NIST. «SP 800-63B: Digital Identity Guidelines: Authentication and Lifecycle Management» [Электронный ресурс] – URL: <https://pages.nist.gov/800-63-3/sp800-63b.html> (дата обращения: 05.10.2025 - 06.02.2026).
13. NIST CSRC. «SP 800-63B (Update 2): Digital Identity Guidelines: Authentication and Lifecycle Management» [Электронный ресурс] – URL: <https://csrc.nist.gov/pubs/sp/800/63/b/upd2/final> (дата обращения: 05.10.2025 - 06.02.2026).
14. Ory. «Ory Hydra Documentation (API Reference)» [Электронный ресурс] – URL: <https://www.ory.com/docs/hydra/reference/api> (дата обращения: 05.10.2025 - 06.02.2026).
15. Ory. «Ory Keto (репозиторий проекта)» [Электронный ресурс] – URL: <https://github.com/ory/keto> (дата обращения: 05.10.2025 - 06.02.2026).
16. Keycloak. «Documentation» [Электронный ресурс] – URL: <https://www.keycloak.org/documentation> (дата обращения: 05.10.2025 - 06.02.2026).
17. Auth0. «Auth0 Docs» [Электронный ресурс] – URL: <https://auth0.com/docs> (дата обращения: 05.10.2025 - 06.02.2026).
18. Auth0. «Authentication API» [Электронный ресурс] – URL: <https://auth0.com/docs/api/authentication/> (дата обращения: 05.10.2025 - 06.02.2026).
19. Firebase. «Firebase Authentication Documentation» [Электронный ресурс] – URL: <https://firebase.google.com/docs/auth> (дата обращения: 05.10.2025 - 06.02.2026).
20. OAuth.net. «OAuth 2.0» [Электронный ресурс] – URL: <https://oauth.net/2/> (дата обращения: 05.10.2025 - 06.02.2026).

ПРИЛОЖЕНИЕ А. Графическая часть выпускной квалификационной работы

В графическую часть выпускной квалификационной работы входят 14 слайдов:

1. Тема ВКР;
2. Определения;
3. Актуальность работы;
4. Постановка задачи;
5. Структура проекта;
6. Механизмы авторизации;
7. Реализация авторизации;
8. Структура урока;
9. Представление формул;
- 10.Ролевая модель;
- 11.Видеоплеер;
- 12.Кодеки;
- 13.Демонстрация работы;
- 14.Заключение.

